

Agatha Barbosa Marinho dos Santos – 1142708666

Bruna Kinjo Luiz Pinto – 1142710312

Kayke Ahrens Biscegli – 1142709883

**Classificação Automática de Chamados de TI
Utilizando Processamento de Linguagem
Natural com Explicabilidade Baseada em Grafos**

São Paulo

2026

Agatha Barbosa Marinho dos Santos – 1142708666

Bruna Kinjo Luiz Pinto – 1142710312

Kayke Ahrens Biscegli – 1142709883

Classificação Automática de Chamados de TI Utilizando Processamento de Linguagem Natural com Explicabilidade Baseada em Grafos

Proposta de Trabalho de Conclusão de Curso
apresentado ao SENAC Santo Amaro como
requisito parcial para aprovação na disciplina
de TCC1 do curso de Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Ciência da Computação

Orientador: Prof. Douglas José Baquião Ribeiro

São Paulo
2026

Resumo

Este documento apresenta a proposta inicial do Trabalho de Conclusão de Curso (TCC1) desenvolvido no SENAC Santo Amaro. O objetivo deste trabalho é investigar se a inclusão de uma camada de explicabilidade baseada em SHAP, LIME e grafos de co-ocorrência a um *pipeline* de classificação com BERTimbau aumenta a transparência e auditabilidade do processo decisório, em relação a abordagens sem explicabilidade, utilizando chamados de suporte técnico em português brasileiro. A justificativa para este estudo baseia-se na necessidade de automatizar a triagem manual de *tickets* em sistemas de *help desk* corporativos, processo atualmente lento, inconsistente e sujeito a erros humanos, ao mesmo tempo que se endereça o problema da "caixa-preta" dos modelos de aprendizado profundo, que dificulta a adoção em produção devido à falta de transparência nas decisões algorítmicas. A revisão bibliográfica abrange sistemas de *help desk* e triagem de chamados, processamento de linguagem natural para classificação de texto, arquitetura *Transformer* e o modelo BERT, inteligência artificial explicável com foco em SHAP e LIME, e grafos de co-ocorrência como estrutura de representação e visualização. A solução prevista consiste em um *pipeline* composto por: um módulo de classificação baseado em BERTimbau *fine-tuned* sobre *dataset* de chamados reais em PT-BR; um módulo de explicabilidade que calcula valores SHAP e *scores* LIME para cada predição; um módulo de visualização que constrói grafos de co-ocorrência ponderados pelos *scores* XAI, permitindo aos analistas de TI auditar visualmente as decisões do modelo. Este trabalho está organizado em três capítulos que apresentam a introdução ao tema, a revisão bibliográfica que fundamenta a pesquisa, o desenvolvimento com a solução proposta e o cronograma de trabalho, seguidos das referências bibliográficas.

Palavras-chave: Processamento de Linguagem Natural. BERT. Inteligência Artificial Explicável. Grafos de Co-ocorrência. Classificação de Tickets de TI.

Lista de ilustrações

Figura 1 – Exemplo de explicação gerada pelo LIME	24
Figura 2 – Representação dos valores SHAP por <i>feature</i>	26
Figura 3 – Protótipo de baixa fidelidade da interface por Agatha Barbosa	40
Figura 4 – Protótipo de baixa fidelidade da interface por Bruna Kinjo	40
Figura 5 – Protótipo de baixa fidelidade da interface por Kayke Ahrens	41
Figura 6 – Protótipo de baixa fidelidade da interface final - Tela do grafo	41
Figura 7 – Protótipo de baixa fidelidade da interface final - Tela das estatísticas	42
Figura 8 – Protótipo de média fidelidade - visualização do grafo de co-ocorrência	43
Figura 9 – Protótipo de média fidelidade - visualização estatística dos <i>tokens</i>	43
Figura 10 – Cronograma de atividades – Diagrama de Gantt	47

Lista de tabelas

Tabela 1 – Comparação entre trabalhos relacionados	36
Tabela 2 – Cenários experimentais para avaliação do sistema	39

Lista de abreviaturas e siglas

BERT	<i>Bidirectional Encoder Representations from Transformers</i>
BERTimbau	Variante do BERT pré-treinada para o português brasileiro
BiLSTM	<i>Bidirectional Long Short-Term Memory</i>
CNN	<i>Convolutional Neural Network</i> (Rede Neural Convolucional)
GCN	<i>Graph Convolutional Network</i> (Rede Neural Convolucional em Grafos)
GloVe	<i>Global Vectors for Word Representation</i>
GNN	<i>Graph Neural Network</i> (Rede Neural de Grafos)
HOOM	<i>Hybrid Online Offline Model</i>
ITSM	<i>IT Service Management</i> (Gerenciamento de Serviços de TI)
JSON	<i>JavaScript Object Notation</i>
LIME	<i>Local Interpretable Model-Agnostic Explanations</i>
LSTM	<i>Long Short-Term Memory</i>
mBERT	<i>Multilingual BERT</i>
MLM	<i>Masked Language Modeling</i>
MLP	<i>Multilayer Perceptron</i> (Perceptron Multicamadas)
NER	<i>Named Entity Recognition</i> (Reconhecimento de Entidades Nomeadas)
NLTK	<i>Natural Language Toolkit</i>
NSP	<i>Next Sentence Prediction</i>
PLN	Processamento de Linguagem Natural
PMI	<i>Pointwise Mutual Information</i> (Informação Mútua Pontual)
PT-BR	Português Brasileiro
RNN	<i>Recurrent Neural Network</i> (Rede Neural Recorrente)
ROC-AUC	<i>Receiver Operating Characteristic — Area Under the Curve</i>
RoBERTa	<i>Robustly Optimized BERT Pretraining Approach</i>

SHAP	<i>SHapley Additive exPlanations</i>
SLA	<i>Service Level Agreement</i> (Acordo de Nível de Serviço)
SMOTE	<i>Synthetic Minority Oversampling Technique</i>
SVM	<i>Support Vector Machine</i> (Máquina de Vetores de Suporte)
TF-IDF	<i>Term Frequency–Inverse Document Frequency</i>
TI	Tecnologia da Informação
XAI	<i>Explainable Artificial Intelligence</i> (Inteligência Artificial Explicável)
XLM-RoBERTa	<i>Cross-lingual Language Model RoBERTa</i>

Sumário

1	INTRODUÇÃO	10
1.1	Contexto	10
1.2	Justificativa	11
1.3	Objetivo	11
1.3.1	Objetivo principal	11
1.3.2	Objetivos secundários	12
2	REVISÃO BIBLIOGRÁFICA	13
2.1	Sistemas de <i>Help Desk</i> e a Triagem de Chamados de TI	13
2.1.1	Definição e importância dos sistemas de <i>help desk</i> nas organizações	13
2.1.2	O processo manual de triagem e seus problemas	13
2.1.3	Abordagens de aprendizado de máquina para classificação automática de <i>tickets</i>	14
2.1.3.1	Modelos clássicos — <i>Naive Bayes</i> e SVM com TF-IDF	14
2.1.3.2	Redes neurais convolucionais aplicadas a <i>tickets</i>	14
2.1.3.3	Classificação de chamados no contexto brasileiro	15
2.1.4	Limitações dos modelos existentes	16
2.1.4.1	O debate BERT versus SVM em domínios especializados	16
2.1.4.2	A caixa-preta como barreira à adoção em produção	16
2.2	Processamento de Linguagem Natural para Classificação de Texto	17
2.2.1	Representações vetoriais de texto	17
2.2.1.1	<i>TF-IDF</i> e modelos baseados em frequência	17
2.2.1.2	<i>Embeddings</i> estáticos — <i>Word2Vec</i> e <i>GloVe</i>	17
2.2.1.3	Limitações das representações estáticas — polissemia e contexto	18
2.3	O Modelo BERT e suas Variantes	18
2.3.1	Arquitetura <i>Transformer</i> e mecanismo de atenção	18
2.3.1.1	<i>Self-attention</i> e atenção bidirecional	18
2.3.1.2	Pré-treinamento — <i>Masked Language Modeling</i> e <i>Next Sentence Prediction</i>	19
2.3.1.3	<i>Fine-tuning</i> para tarefas <i>downstream</i>	20
2.3.2	BERT	20
2.3.2.1	Arquitetura e configurações <i>BERT-base</i> e <i>BERT-large</i>	20
2.3.2.2	<i>Token</i> [CLS] e classificação de sequências	20
2.3.3	BERTimbau — BERT para o português brasileiro	21
2.3.3.1	Corpus de pré-treinamento e diferenças em relação ao mBERT	21
2.3.3.2	Desempenho em tarefas de PLN em PT-BR	21

2.3.4	<i>Fine-tuning</i> de BERT para classificação de chamados de TI	22
2.4	Inteligência Artificial Explicável (XAI)	22
2.4.1	Definição e motivação	22
2.4.1.1	O problema da caixa-preta em aprendizado profundo	22
2.4.1.2	Confiança, auditabilidade e adoção em produção	23
2.4.1.3	Taxonomia de métodos XAI — intrínsecos versus <i>post-hoc</i> , locais versus globais	23
2.4.2	LIME — <i>Local Interpretable Model-Agnostic Explanations</i>	23
2.4.2.1	Funcionamento — perturbações e modelo local interpretável	23
2.4.2.2	Aplicação em classificação de texto	25
2.4.2.3	Vantagens e limitações	25
2.4.3	SHAP — <i>SHapley Additive exPlanations</i>	25
2.4.3.1	Fundamentos — valores de Shapley da teoria dos jogos	25
2.4.3.2	Propriedades matemáticas — eficiência, simetria e ausência	26
2.4.3.3	<i>KernelSHAP</i> e aplicação em modelos de texto	26
2.4.4	Comparação entre SHAP e LIME	27
2.4.4.1	Consistência e estabilidade das explicações	27
2.4.4.2	Custo computacional	27
2.4.4.3	Justificativa para uso conjunto neste trabalho	27
2.4.5	XAI aplicado a classificação de texto e tickets de TI	28
2.4.5.1	LIME em classificação multiclasse de texto	28
2.4.5.2	SHAP e LIME em ambiente ITSM corporativo	28
2.5	Grafos de Co-ocorrência como Representação Estrutural de Texto	29
2.5.1	Fundamentos de grafos aplicados a texto	29
2.5.1.1	Definição de grafo de co-ocorrência — nós, arestas e janela deslizante	29
2.5.1.2	Ponderação de arestas — frequência, PMI	29
2.5.2	<i>TextGCN</i> — formalização do grafo de co-ocorrência	30
2.5.2.1	Grafo heterogêneo documento-palavra	30
2.5.2.2	Redes neurais de grafos e propagação de informação	30
2.5.3	Combinação de BERT com grafos de co-ocorrência	30
2.5.3.1	<i>Embeddings</i> BERT como <i>features</i> iniciais dos nós	30
2.5.3.2	Resultados em vocabulário técnico especializado	31
2.5.4	Grafo de co-ocorrência combinado com XAI	31
2.5.4.1	Ponderação de nós por <i>scores</i> de explicabilidade	31
2.5.4.2	Visualização interativa como camada de auditabilidade	32
2.6	Síntese da Literatura e Posicionamento do Trabalho	32
2.6.1	Lacunas identificadas no estado da arte	32
2.6.1.1	Ausência de XAI em sistemas de classificação de tickets em produção	32
2.6.1.2	Carência de soluções robustas para PT-BR em <i>helpdesk</i>	32
2.6.1.3	Custo-benefício do BERT sem camada de explicabilidade	32

2.6.2	Direcionamento para a proposta	33
2.7	Metodologia da pesquisa bibliográfica	33
2.8	Fundamentos	33
2.8.1	Processamento de Linguagem Natural	33
2.8.2	Modelos de Linguagem Baseados em <i>Transformer</i>	34
2.8.3	Inteligência Artificial Explicável	34
2.8.4	Grafo de Co-ocorrência	34
2.8.5	Sistema de <i>Help Desk</i> e Triagem de Chamados	34
2.9	Trabalhos relacionados	35
2.9.1	Trabalho 1: Ticket Automation: An Insight into Current Research	35
2.9.2	Trabalho 2: Machine Learning for Classification of IT Support Tickets	35
2.9.3	Trabalho 3: Classificação hierárquica de textos aplicado ao contexto chama- dos de suporte	35
2.10	Hipótese de Pesquisa	36
2.11	Métricas de Avaliação	36
3	DESENVOLVIMENTO	38
3.1	Solução proposta	38
3.1.0.1	Visão geral da solução	38
3.1.1	Protocolo experimental	38
3.1.1.1	Protótipos da interface	39
3.1.1.2	Arquitetura da solução	43
3.1.1.3	Tecnologias e ferramentas	45
3.1.1.4	Funcionalidades principais	45
3.1.1.5	Metodologia de implementação	46
3.1.1.6	<i>Dataset</i> e <i>fine-tuning</i>	46
3.1.1.7	Critérios de validação	46
3.2	Cronograma de trabalho	47
	REFERÊNCIAS	48

1 Introdução

A gestão eficiente de *tickets* de suporte técnico em organizações de TI enfrenta desafios crescentes devido ao volume elevado de chamados e à complexidade semântica dos textos gerados por usuários não técnicos (ZEMP, 2021; RAZMA; JURKUS, 2026). Este trabalho investiga o uso de um *pipeline* de classificação automática utilizando Processamento de Linguagem Natural (PLN) com modelos BERT (*Bidirectional Encoder Representations from Transformers*, modelos de linguagem baseados em redes neurais profundas), técnicas de explicabilidade SHAP (*SHapley Additive exPlanations*) e LIME (*Local Interpretable Model-Agnostic Explanations*) — métodos que identificam quais termos do texto influenciaram a decisão do modelo — e visualização em grafos de co-ocorrência (estruturas que representam relações entre termos do texto) para tornar transparente o processo decisório.

1.1 Contexto

Os *helpdesks* corporativos processam diariamente milhares de *tickets* com descrições variadas que misturam jargão técnico, erros ortográficos e expressões coloquiais (ZEMP, 2021). Isso sobrecarrega os analistas, tornando a triagem manual lenta, inconsistente e propensa a erros.

Estudos recentes demonstram que sistemas automatizados com *Transformers* alcançam até 92% de acurácia em classificação multi-categoria (ZEMP, 2021), mas sua natureza de “caixa-preta”, modelos que produzem previsões sem revelar os critérios utilizados para tomá-las, gera resistência entre equipes operacionais que precisam confiar nas decisões algorítmicas para roteamento (RAZMA; JURKUS, 2026).

A raiz matemática dessa intransparência está no mecanismo central desses modelos. A cada camada de processamento, o modelo realiza operações do tipo:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

onde Q , K e V são matrizes de consulta, chave e valor derivadas da entrada (VASWANI et al., 2017). Essas transformações se repetem em centenas de milhões de parâmetros distribuídos em múltiplas camadas, tornando impossível rastrear por que uma palavra específica influenciou determinada classificação. Embora o modelo produza resultados precisos, o caminho percorrido até a decisão permanece completamente não auditável, o que justifica a necessidade de técnicas de explicabilidade como SHAP e LIME para atribuir, de forma matematicamente justificável, a importância de cada termo na previsão (LUNDBERG; LEE, 2017; RIBEIRO; SINGH; GUESTRIN, 2016).

No contexto brasileiro, onde grande parte da comunicação técnica ocorre em português, modelos tradicionais como o *Naive Bayes*, que classifica textos com base em probabilidades, e redes MLP (*Multilayer Perceptron*), que são redes neurais simples sem capacidade de entender o contexto das palavras, apresentam limitações significativas ao lidar com *tickets* reais, pois não conseguem capturar a variabilidade semântica nem o vocabulário técnico específico do domínio de TI (YOKOTA, 2025).

1.2 Justificativa

A triagem de chamados em sistemas de *Help Desk* ainda é frequentemente realizada manualmente por analistas de suporte, que precisam interpretar descrições textuais de problemas e direcionar os chamados para a equipe técnica responsável. Esse processo pode ser lento, sujeito a inconsistências e dependente da experiência individual dos analistas.

Técnicas de Processamento de Linguagem Natural (PLN) e aprendizado de máquina têm sido utilizadas para automatizar essa tarefa por meio da classificação automática de *tickets*. No entanto, muitos desses modelos funcionam como caixas-pretas, dificultando a compreensão de como as decisões são tomadas.

A área de *Explainable Artificial Intelligence* (XAI) busca justamente aumentar a transparência desses sistemas, permitindo compreender quais fatores influenciam as decisões dos modelos. Nesse contexto, a utilização de modelagem em grafos pode oferecer uma forma visual e estrutural de representar relações entre termos e evidenciar como essas relações contribuem para a classificação.

Dessa forma, investigar se mecanismos de explicabilidade baseados em grafos podem tornar o processo decisório mais compreensível representa uma contribuição relevante tanto para aplicações práticas de suporte técnico quanto para o estudo de transparência em sistemas de inteligência artificial.

1.3 Objetivo

Este trabalho estrutura-se a partir de um objetivo central, que define o escopo principal da pesquisa, e de objetivos específicos que representam as metas intermediárias e os subprodutos analíticos necessários para responder à questão central de investigação.

1.3.1 Objetivo principal

Investigar se a inclusão de uma camada de explicabilidade e grafos de co-ocorrência, a um *pipeline* de classificação baseado em *transformers*, aumenta a transparência e auditabilidade do processo decisório, em relação a abordagens sem explicabilidade, utilizando chamados de suporte de TI em português brasileiro.

1.3.2 Objetivos secundários

1. Analisar o ganho de desempenho do modelo BERTimbau com *fine-tuning* em relação ao *baseline* SVM com TF-IDF, utilizando a métrica F1-macro.
2. Comparar a explicabilidade gerada por SHAP e LIME em termos de consistência, estabilidade e relevância semântica das explicações produzidas para chamados de suporte técnico em português brasileiro.
3. Analisar se a visualização em grafos de co-ocorrência ponderados por valores SHAP e LIME fornece auditabilidade adicional ao processo de classificação em relação à exibição isolada de *scores* de importância.
4. Validar a consistência semântica das explicações geradas pelo *pipeline* no contexto de chamados de suporte técnico em português brasileiro, verificando se os termos destacados correspondem ao vocabulário técnico esperado para cada categoria classificada.

2 Revisão Bibliográfica

Esta seção apresenta os fundamentos conceituais que sustentam a proposta deste trabalho. Os temas foram organizados em seis blocos para apresentar a evolução do problema até chegar às limitações identificadas na literatura e à proposta deste trabalho: (1) Sistemas de *Help Desk* e a triagem de chamados de TI; (2) Processamento de Linguagem Natural para classificação de texto; (3) o modelo BERT e suas variantes; (4) Inteligência Artificial Explicável (XAI); (5) grafos de co-ocorrência como representação estrutural de texto; e (6) a síntese da literatura e posicionamento do trabalho.

2.1 Sistemas de *Help Desk* e a Triagem de Chamados de TI

2.1.1 Definição e importância dos sistemas de *help desk* nas organizações

Sistemas de *help desk*, também denominados sistemas de suporte técnico, centrais de serviço ou plataformas ITSM (*IT Service Management*), são estruturas organizacionais e tecnológicas responsáveis por centralizar e gerenciar as solicitações de suporte submetidas pelos usuários de uma organização. Esses sistemas recebem chamados relacionados a falhas de infraestrutura, problemas em aplicações, solicitações de acesso, dúvidas operacionais e demais ocorrências que interrompem ou dificultam o trabalho dos colaboradores (ZEMP, 2021).

A importância desses sistemas para a continuidade operacional das organizações é amplamente reconhecida na literatura. Razma e Jurkus (2026) destacam que empresas modernas geram milhares de chamados de suporte mensalmente, e que cada chamado deve ser corretamente categorizado e encaminhado à equipe responsável para que o prazo de resolução estabelecido nos acordos de nível de serviço (SLA, do inglês *Service Level Agreement*) seja cumprido. O não cumprimento desses prazos impacta diretamente a produtividade organizacional e a satisfação dos usuários.

2.1.2 O processo manual de triagem e seus problemas

Em grande parte das organizações, a triagem de chamados ainda é realizada manualmente por analistas de suporte, que leem a descrição textual do problema submetida pelo usuário, determinam a categoria correta e encaminham o chamado à equipe técnica responsável (ZEMP, 2021). Esse processo apresenta limitações estruturais que se tornam mais evidentes à medida que o volume de chamados cresce.

A inconsistência na categorização é um dos principais problemas. Analistas diferentes podem categorizar o mesmo tipo de chamado de formas distintas, e um mesmo analista

pode ser inconsistente ao longo do tempo. Além disso, a triagem manual depende do conhecimento individual de cada analista sobre a estrutura de categorias da organização, conhecimento que não é facilmente transferível e que se perde quando há rotatividade de pessoal (WAHBA, 2023).

Chamados categorizados incorretamente são encaminhados para equipes que não possuem competência para resolvê-los, isso gera um ciclo de redistribuição de chamados que eleva o tempo de resolução, desperdiça o tempo da equipe e deixa o usuário frustrado (HARUN; HUSPI; IAHAD, 2023). Pereira et al. (2023) observam que, no contexto brasileiro, esse problema é agravado pela informalidade da linguagem utilizada pelos usuários ao descrever seus problemas, o que dificulta ainda mais a categorização manual.

2.1.3 Abordagens de aprendizado de máquina para classificação automática de *tickets*

A automação da triagem de chamados por meio de aprendizado de máquina tem sido investigada de forma crescente na literatura. O problema é tratado como uma tarefa de classificação supervisionada de texto: dado o histórico de chamados já categorizados, treina-se um modelo capaz de prever a categoria de novos chamados a partir de sua descrição textual.

2.1.3.1 Modelos clássicos — *Naive Bayes* e SVM com TF-IDF

As primeiras abordagens automatizadas para classificação de chamados utilizaram algoritmos clássicos de aprendizado de máquina combinados com representações baseadas em frequência de termos. Harun, Huspi e Iahad (2023) investigaram a classificação automática de perguntas em um sistema de *help desk* universitário, comparando *Naive Bayes* e SVM (*Support Vector Machine*), um algoritmo que classifica textos encontrando a melhor separação possível entre as categorias, com vetorização TF-IDF. Os resultados demonstraram que o SVM com TF-IDF supera o *Naive Bayes* em acurácia e F1-score (métrica que combina precisão e revocação em um único valor, detalhada na Seção 2.11), estabelecendo-se como referência para abordagens clássicas nesse domínio.

Apesar de sua eficiência computacional e facilidade de interpretação, esses modelos apresentam limitações na captura de relações semânticas entre termos. A representação TF-IDF trata cada palavra de forma independente, sem considerar o contexto em que ela aparece, o que restringe a capacidade do modelo de distinguir categorias semanticamente próximas.

2.1.3.2 Redes neurais convolucionais aplicadas a *tickets*

Com o avanço das técnicas de aprendizado profundo, modelos baseados em redes neurais convolucionais (CNN) passaram a ser aplicados à classificação de chamados de

suporte. [Paramesh e Shreedhara \(2022\)](#) propuseram um classificador baseado em CNN com *word embeddings* (representações vetoriais densas de palavras aprendidas por redes neurais) para categorização automática de *tickets* de *service desk* de infraestrutura de TI. Os autores compararam o modelo proposto com SVM, *Naive Bayes*, Regressão Logística e *K-Nearest Neighbor*, demonstrando que a CNN supera todos os modelos clássicos em acurácia e F1-score, especialmente em categorias com descrições textuais mais longas e complexas.

A vantagem das CNN em relação aos modelos clássicos reside na capacidade de identificar automaticamente quais grupos de palavras que aparecem juntas são importantes para a classificação, sem que um especialista precise definir isso manualmente. Essa progressão de modelos lineares com TF-IDF para redes profundas com *embeddings*, estabelece o contexto histórico sobre o qual os modelos baseados em *transformers* representam um novo salto qualitativo.

2.1.3.3 Classificação de chamados no contexto brasileiro

A literatura sobre classificação automática de chamados de suporte em português brasileiro ainda é escassa, e os trabalhos existentes evidenciam limitações importantes das abordagens mais simples nesse contexto específico.

[Pereira et al. \(2023\)](#) apresentaram um modelo de classificação automática de chamados de suporte de TI em sete categorias, atingindo 75% de precisão média com modelos de aprendizado de máquina tradicionais. Os autores disponibilizaram publicamente o código, o modelo e os dados anonimizados, e desenvolveram um protótipo *web* com *backend* e *frontend* para uso dos analistas de TI, tornando-o um dos poucos trabalhos nacionais com *dataset* público e interface funcional. No entanto, a acurácia de 75% deixa margem considerável para melhoria, especialmente em cenários com maior número de categorias ou com classes semanticamente próximas.

[Yokota \(2025\)](#) investigou a classificação hierárquica de chamados de suporte utilizando *Naive Bayes* e *Multilayer Perceptron* (MLP) com duas variações arquiteturais. Os resultados, avaliados por F1-macro para lidar com o desbalanceamento entre classes, foram insatisfatórios, evidenciando que modelos simples falham em capturar as nuances semânticas presentes em textos técnicos de suporte em PT-BR. A autora conclui que a abordagem proposta revelou desafios substanciais na classificação automática de textos especializados, evidenciando a necessidade de abordagens metodológicas mais sofisticadas, o que reforça a escolha do BERTimbau neste trabalho.

2.1.4 Limitações dos modelos existentes

2.1.4.1 O debate BERT versus SVM em domínios especializados

Wahba (2023), em sua tese de doutorado na Universidade de Western Ontario, propõe um contraponto relevante ao uso indiscriminado de modelos baseados em *transformers* para classificação de texto em domínios especializados. Com base em quatro anos de estudos empíricos com dados reais de uma empresa parceira, a autora demonstra que chamados de suporte de TI utilizam vocabulário técnico específico e de baixa ambiguidade. Nesse contexto, a principal vantagem do BERT, que é compreender palavras com múltiplos sentidos dependendo do contexto, tem impacto reduzido, tornando o SVM com TF-IDF competitivo em termos de desempenho.

Com base nessa observação, Wahba propõe o modelo HOOM (*Hybrid Online Offline Model*), que combina um classificador SVM linear treinado em modo *offline* com um classificador *Passive Aggressive Classifier*. Esse segundo classificador é um algoritmo de aprendizado incremental que atualiza o modelo a cada nova amostra, mantendo-se “passivo” quando a predição está correta e “agressivo” ao corrigir erros e treinado em modo *online* para detectar mudanças na distribuição dos dados ao longo do tempo. O modelo é descrito como mais eficiente, interpretável e reproduzível do que abordagens baseadas em *transformers*, que são modelos de linguagem baseados em redes neurais profundas e que serão detalhados na Seção 2.3.

Este argumento é diretamente relevante para o presente trabalho, pois o TCC responde à questão levantada por Wahba: o custo computacional adicional do BERTimbau se justifica quando se adiciona uma camada de XAI e grafo de co-ocorrência que transforma a saída pouco interpretável do modelo em algo auditável pelos analistas. Em outras palavras, a interpretabilidade não é apenas uma funcionalidade adicional, é a justificativa central para a escolha do modelo mais complexo.

2.1.4.2 A caixa-preta como barreira à adoção em produção

Zemp (2021), em estudo desenvolvido na ZHAW com dados reais de um banco suíço de médio porte, treinou três tipos de modelos: CNN, BiLSTM, que é uma rede neural recorrente bidirecional capaz de processar sequências de texto nos dois sentidos, e RoBERTa, uma variante aprimorada do BERT com treinamento mais robusto. O estudo utilizou mais de um milhão de *tickets* históricos, atingindo 92% de acurácia de validação e F1-macro de 0,75 em mais de 300 classes. O sistema foi parcialmente implantado em produção, e os agentes de suporte foram convidados a utilizá-lo em seu trabalho diário. O estudo revelou que, apesar dos resultados técnicos expressivos, os analistas resistiram à adoção do sistema na prática. A principal razão relatada foi a ausência de justificativas para as predições: os agentes não conseguiam confiar em um sistema que não explicava por que havia escolhido determinada categoria.

Razma e Jurkus (2026) corroboram essa observação no contexto de um ambiente ITSM corporativo multilíngue com mais de 87 mil *tickets*. Os autores aplicaram LIME e SHAP para gerar explicações das predições do mBERT (multilingual BERT), uma versão do BERT treinada simultaneamente em mais de 100 idiomas, e do modelo de regressão logística, e concluíram que a camada de interpretabilidade aumentou a confiança dos operadores no sistema, mesmo nos casos em que a predição estava incorreta pois o analista conseguia identificar por que o modelo havia errado.

Esses estudos ajudam a justificar a proposta deste trabalho: sistemas de classificação de chamados de TI em produção real funcionam, mas a ausência de mecanismos de explicabilidade limita sua adoção e auditabilidade. A proposta deste TCC busca lidar com essa limitação.

2.2 Processamento de Linguagem Natural para Classificação de Texto

2.2.1 Representações vetoriais de texto

2.2.1.1 *TF-IDF* e modelos baseados em frequência

A representação mais simples de um documento para fins de classificação é o modelo *bag-of-words*: cada documento é representado como um vetor de contagens de palavras, sem considerar a ordem em que aparecem. O *TF-IDF* é um refinamento desse modelo que pondera cada termo pelo produto de sua frequência no documento e pelo inverso de sua frequência no corpus, reduzindo o peso de termos muito comuns e aumentando o peso de termos mais informativos (RAZMA; JURKUS, 2026).

Apesar de sua simplicidade, o *TF-IDF* ainda é amplamente utilizado como *baseline* em tarefas de classificação de texto, especialmente quando combinado com SVM, e apresenta desempenho competitivo em domínios com vocabulário técnico específico e baixa polissemia, ou seja, onde as palavras raramente mudam de sentido dependendo do contexto (WAHBA, 2023). Sua principal limitação é a incapacidade de capturar relações semânticas entre termos: palavras sinônimas ou semanticamente relacionadas são tratadas como completamente distintas.

2.2.1.2 *Embeddings* estáticos — *Word2Vec* e *GloVe*

Os *embeddings* de palavras representam um avanço significativo em relação às abordagens baseadas em frequência, ao mapear cada palavra para um vetor denso em um espaço de baixa dimensionalidade, de forma que palavras semanticamente relacionadas sejam representadas por vetores próximos. O *Word2Vec*, proposto por Mikolov et al. (2013), aprende *embeddings* a partir de grandes corpora (plural de *corpus*, conjunto amplo de

textos usado para treinamento) textuais utilizando redes neurais rasas, treinadas para prever palavras a partir de seu contexto ou contexto a partir de palavras.

O GloVe (*Global Vectors for Word Representation*) utiliza estatísticas globais de co-ocorrência do corpus para produzir representações com propriedades algébricas notáveis: vetores de palavras semanticamente relacionadas tendem a ser próximos no espaço vetorial, e operações aritméticas entre vetores refletem relações semânticas — o exemplo clássico é que o vetor de “rei” menos “homem” mais “mulher” resulta em um vetor próximo ao de “rainha”.

Esses *embeddings* estáticos representaram um avanço expressivo na qualidade das representações textuais e foram amplamente adotados como camada de entrada em modelos de aprendizado profundo para PLN. No entanto, eles atribuem um único vetor a cada palavra, independentemente do contexto em que ela aparece, limitação conhecida como o problema da representação estática.

2.2.1.3 Limitações das representações estáticas — polissemia e contexto

O grande problema dos *embeddings* estáticos é que eles não conseguem identificar quando uma palavra muda de sentido dependendo do contexto — como “banco”, que pode ser um assento ou uma instituição financeira. Em textos técnicos de suporte de TI, isso se manifesta em termos como “acesso”, “serviço” ou “sistema”, que podem ter significados muito distintos dependendo do chamado.

Essa limitação motivou o desenvolvimento de representações contextualizadas, nas quais o *embedding* de cada palavra é computado dinamicamente a partir do contexto completo da frase. Os modelos baseados na arquitetura *Transformer*, em particular o BERT, representam a abordagem dominante nessa direção (DEVLIN et al., 2019).

2.3 O Modelo BERT e suas Variantes

2.3.1 Arquitetura *Transformer* e mecanismo de atenção

2.3.1.1 *Self-attention* e atenção bidirecional

A arquitetura *Transformer* é composta por dois blocos principais: um *encoder* (codificador) e um *decoder* (decodificador), cada um formado por camadas empilhadas (VASWANI et al., 2017). O *encoder* é responsável por receber a sequência de entrada e transformá-la em representações vetoriais ricas em informação contextual. O *decoder*, por sua vez, utiliza essas representações para gerar a sequência de saída como uma tradução ou resposta. Cada camada do *encoder* é composta por dois sub-componentes principais: o mecanismo de *self-attention*, que permite que cada *token*, ou seja, cada unidade mínima de texto em que o modelo divide a entrada, “observe” todos os outros da sequência, e uma rede neural densa aplicada posição a posição (*feed-forward network*). O resultado de cada sub-componente

passa por uma operação de normalização (*layer normalization*) que estabiliza o treinamento (VASWANI et al., 2017). Vale destacar que modelos como o BERT utilizam apenas o bloco *encoder*, pois seu objetivo é produzir representações contextuais do texto de entrada, e não gerar novas sequências (DEVLIN et al., 2019).

A arquitetura *Transformer*, proposta por Vaswani et al. (2017), introduziu o mecanismo de *self-attention* como alternativa às redes recorrentes para processamento de sequências. As RNN (*Recurrent Neural Networks*) são uma classe de redes neurais projetadas para processar dados sequenciais, nos quais o contexto e a ordem temporal importam, processando os *tokens* um por vez, da esquerda para a direita. As LSTM (*Long Short-Term Memory*) são um tipo especializado de RNN projetado para aprender dependências de longo prazo em sequências, mitigando o problema de degradação do gradiente presente nas RNN simples. Apesar dessas melhorias, ambas as arquiteturas ainda processam a sequência de forma estritamente sequencial, o que limita o paralelismo computacional e dificulta a captura de relações entre termos distantes.

Para compreender o mecanismo de atenção, considere a frase “o banco bloqueou meu acesso”: para determinar o significado da palavra “banco”, é necessário considerar as demais palavras da frase — especialmente “bloqueou” e “acesso” — que indicam tratar-se de uma instituição financeira e não de um assento. O *self-attention* formaliza esse processo: para cada *token* da sequência, o mecanismo calcula um score de relevância em relação a todos os outros *tokens*, indicando o quanto cada um deve “influenciar” a representação do *token* atual. “Atender simultaneamente a todas as posições” significa que esse cálculo é realizado em paralelo para todos os pares de *tokens* da sequência, possibilitando a captura de dependências de longo alcance de forma muito mais eficiente do que as arquiteturas recorrentes.

Modelos como o GPT utilizam atenção unidirecional (cada *token* só pode atender aos *tokens* anteriores). O BERT, por sua vez, utiliza atenção bidirecional: cada *token* atende simultaneamente a todos os outros *tokens* da sequência, tanto os que aparecem antes quanto os que aparecem depois. Essa bidirecionalidade é fundamental para a qualidade das representações contextualizadas geradas pelo modelo, uma vez que o significado de uma palavra frequentemente depende tanto de seu contexto anterior quanto posterior (DEVLIN et al., 2019).

2.3.1.2 Pré-treinamento — *Masked Language Modeling* e *Next Sentence Prediction*

O BERT é pré-treinado em dois objetivos complementares sobre grandes corpora textuais não rotulados. O primeiro, *Masked Language Modeling* (MLM), mascara aleatoriamente 15% dos *tokens* de cada sequência de entrada e treina o modelo para prever os *tokens* mascarados a partir do contexto bidirecional restante. Esse objetivo força o modelo a aprender representações ricas que capturam o significado de cada palavra em seu contexto completo (DEVLIN et al., 2019).

O segundo objetivo, *Next Sentence Prediction* (NSP), treina o modelo a determinar se duas sentenças apresentadas em sequência são consecutivas no texto original ou foram pareadas aleatoriamente. Esse objetivo permite ao modelo aprender relações semânticas entre sentenças, o que é útil para tarefas como inferência textual e resposta a perguntas.

2.3.1.3 *Fine-tuning* para tarefas *downstream*

Após o pré-treinamento, o BERT pode ser adaptado para tarefas específicas por meio do processo de *fine-tuning*: o modelo pré-treinado é inicializado com os pesos aprendidos no pré-treinamento, uma camada de saída específica para a tarefa é adicionada ao topo da arquitetura, e o modelo completo é treinado por um número reduzido de épocas sobre o conjunto de dados rotulado da tarefa-alvo (DEVLIN et al., 2019).

Essa abordagem de pré-treinamento seguido de *fine-tuning* é particularmente eficaz porque o modelo já adquiriu conhecimento linguístico geral durante o pré-treinamento, e o *fine-tuning* apenas adapta esse conhecimento para o domínio e a tarefa específicos. Para classificação de chamados de TI, o *fine-tuning* ajusta os pesos do modelo para que as representações produzidas sejam especialmente discriminativas entre as categorias de chamados presentes no *dataset*.

2.3.2 BERT

2.3.2.1 Arquitetura e configurações *BERT-base* e *BERT-large*

Devlin et al. (2019) propuseram o BERT em duas configurações. O *BERT-base* possui 12 camadas de *encoder Transformer*, 12 cabeças de atenção e 110 milhões de parâmetros. O *BERT-large* possui 24 camadas, 16 cabeças de atenção e 340 milhões de parâmetros. Ambas as versões foram pré-treinadas sobre o corpus da Wikipedia em inglês e o BookCorpus, totalizando aproximadamente 3,3 bilhões de *tokens*.

O BERT estabeleceu o estado da arte em onze tarefas de PLN no momento de sua publicação, incluindo classificação de texto, reconhecimento de entidades nomeadas, resposta a perguntas e inferência textual. Seu lançamento popularizou o uso de modelos pré-treinados em tarefas de PLN, consolidando a abordagem de pré-treinamento em larga escala seguido de *fine-tuning* como a estratégia dominante para tarefas de PLN.

2.3.2.2 *Token [CLS]* e classificação de sequências

Para tarefas de classificação de sequências, o BERT utiliza um *token* especial [CLS] (abreviação de *Classification*, marcador inserido pelo modelo para agregar a representação global da sequência) inserido no início de cada sequência de entrada. Após o processamento pelas camadas de *encoder*, a representação vetorial correspondente ao [CLS] agrega informação contextual de toda a sequência e é usada como entrada para a camada de classificação (DEVLIN et al., 2019). Durante o *fine-tuning*, essa camada de classificação,

tipicamente uma camada *linear* seguida de *softmax*, aprende a mapear a representação do [CLS] para as probabilidades de cada categoria. O presente trabalho pretende usar essa representação para classificar o chamado de TI em uma das categorias predefinidas, e os valores SHAP e LIME serão calculados sobre os tokens de entrada para identificar quais contribuíram mais para a predição.

2.3.3 BERTimbau — BERT para o português brasileiro

2.3.3.1 Corpus de pré-treinamento e diferenças em relação ao mBERT

Souza, Nogueira e Lotufo (2020) desenvolveram o BERTimbau, modelo BERT pré-treinado especificamente para o português brasileiro, a partir de um corpus de 2,68 bilhões de *tokens* composto por textos da Wikipedia em português, do BrWaC (*Brazilian Web as Corpus*) e de outros corpora nacionais. Duas versões foram disponibilizadas: BERTimbau-base (12 camadas, 110M parâmetros) e BERTimbau-large (24 camadas, 330M parâmetros).

A principal diferença em relação ao mBERT está no foco linguístico: o mBERT foi pré-treinado sobre 104 idiomas simultaneamente, o que dilui a capacidade de representação para cada língua individualmente. O BERTimbau, ao ser pré-treinado exclusivamente sobre texto em português brasileiro, aprende padrões morfológicos, sintáticos e semânticos específicos do idioma com maior profundidade.

2.3.3.2 Desempenho em tarefas de PLN em PT-BR

Souza, Nogueira e Lotufo (2020) avaliaram o BERTimbau em três tarefas de PLN em português: reconhecimento de entidades nomeadas (NER), análise de sentimento e similaridade textual. Em todas as tarefas, o BERTimbau superou o mBERT por margem significativa, consolidando-se como o modelo de referência para aplicações de PLN em PT-BR.

Para o presente trabalho, a escolha do BERTimbau em detrimento do mBERT é motivada por dois fatores. Primeiro, os chamados de suporte analisados estão em português brasileiro, idioma para o qual o BERTimbau possui representações mais ricas. Segundo, os chamados utilizam terminologia técnica de TI em português, que pode diferir substancialmente de sua versão em inglês. E o BERTimbau, pré-treinado em corpus nacional, tem maior probabilidade de ter encontrado esse vocabulário durante o pré-treinamento.

Vale destacar que o BERTimbau permanece como escolha principal deste trabalho, dado que os chamados de suporte a serem analisados estão predominantemente em português brasileiro. No entanto, caso a análise exploratória do *dataset* a ser utilizado revele uma proporção significativa de termos técnicos em inglês, a escolha do modelo será reavaliada empiricamente durante o TCC2, podendo considerar modelos multilíngues como o mBERT ou o XLM-RoBERTa.

2.3.4 *Fine-tuning* de BERT para classificação de chamados de TI

A aplicação de BERT e modelos derivados à classificação de chamados de TI tem demonstrado resultados expressivos na literatura recente. [Borges \(2024\)](#) realizou uma comparação sistemática entre BERT e algoritmos clássicos, SVM e *Naive Bayes*, em tarefas de classificação de texto em português e inglês, utilizando quatro *datasets* públicos. Os resultados confirmaram que o BERT supera consistentemente os modelos tradicionais em acurácia e F1, especialmente em tarefas multiclasse com nuances semânticas mais complexas, embora com custo computacional consideravelmente maior.

[Zemp \(2021\)](#) realizou o *fine-tuning* de RoBERTa, uma variante do BERT com treinamento aprimorado, sobre mais de um milhão de *tickets* reais de um banco suíço, atingindo F1-macro de 0,75 em mais de 300 classes. O autor demonstra que *transformers* superam CNN e BiLSTM nesse domínio, mas aponta que a incapacidade do modelo de explicar suas decisões representa uma barreira à adoção em produção.

[Razma e Jurkus \(2026\)](#) compararam regressão logística com TF-IDF frente a mBERT e XLM-RoBERTa, uma variante do RoBERTa desenvolvida especificamente para lidar com múltiplos idiomas simultaneamente, em um corpus de 87 mil *tickets* multilíngues de uma empresa de manufatura. Os resultados confirmaram que *transformers* superam modelos clássicos em macro-F1, especialmente para classes de baixa frequência e semanticamente complexas. A combinação de *fine-tuning* com LIME e SHAP produziu explicações qualitativamente mais ricas do que o modelo linear, validando o uso conjunto de BERT e XAI em ambiente ITSM de produção.

2.4 Inteligência Artificial Explicável (XAI)

2.4.1 Definição e motivação

2.4.1.1 O problema da caixa-preta em aprendizado profundo

Modelos de aprendizado profundo, incluindo o BERT, são frequentemente descritos como caixas-pretas: produzem predições com alta acurácia, mas sem revelar quais características da entrada determinaram a saída. Isso acontece porque esses modelos possuem centenas de milhões de parâmetros internos, valores numéricos ajustados durante o treinamento, organizados em múltiplas camadas de cálculos complexos. Essa estrutura torna impossível para um ser humano acompanhar ou explicar, passo a passo, como o modelo chegou a uma determinada decisão ([SALIH et al., 2023](#)).

Em contextos como a triagem de chamados de TI, essa opacidade representa um problema prático concreto. Analistas de suporte e gestores precisam auditar as decisões do sistema, identificar erros sistemáticos e eventualmente contestar categorizações incorretas. Um modelo que não fornece justificativa para suas predições é difícil de depurar, difícil de

melhorar e difícil de justificar perante *stakeholders* (partes interessadas, como gestores e diretores) organizacionais (ZEMP, 2021).

2.4.1.2 Confiança, auditabilidade e adoção em produção

A relação entre explicabilidade e confiança em sistemas de IA é documentada empiricamente na literatura. Zemp (2021) relata que agentes de suporte demonstraram resistência a adotar o sistema de classificação, especificamente porque poderia levar a decisões automáticas sem análise adequada. Razma e Jurkus (2026) observam que a adição de explicações LIME e SHAP ao sistema aumentou a disposição dos operadores a aceitar as previsões, mesmo em casos de incerteza, porque o analista conseguia verificar se os termos destacados como mais influentes faziam sentido para aquela categoria específica.

Esse fenômeno é consistente com a literatura de XAI mais ampla: a explicabilidade não melhora necessariamente a acurácia do modelo, mas aumenta a confiança dos usuários no sistema e facilita a identificação de falhas sistemáticas, o que é essencial para a adoção sustentada em ambientes de produção.

2.4.1.3 Taxonomia de métodos XAI — intrínsecos versus *post-hoc*, locais versus globais

Os métodos de XAI podem ser classificados segundo duas dimensões principais (SALIH et al., 2023). A primeira dimensão distingue métodos intrínsecos dos métodos *post-hoc*. Métodos intrínsecos são aqueles em que a interpretabilidade é uma propriedade do próprio modelo — árvores de decisão, regressão logística e modelos lineares em geral são interpretáveis por *design*, pois seus parâmetros têm significado diretamente interpretável. Métodos *post-hoc* são aplicados após o treinamento, sobre modelos de qualquer complexidade, e produzem explicações sem modificar o modelo original.

A segunda dimensão distingue explicações locais das globais. Explicações locais descrevem o comportamento do modelo para uma instância específica de entrada (exemplo: por que o modelo classificou *este* chamado como “Acesso?”). Explicações globais descrevem o comportamento geral do modelo (exemplo: quais termos são mais importantes para cada categoria em média?). SHAP e LIME são métodos *post-hoc* que operam primariamente no nível local, embora o SHAP permita agregações que produzem explicações globais.

2.4.2 LIME — *Local Interpretable Model-Agnostic Explanations*

2.4.2.1 Funcionamento — perturbações e modelo local interpretável

LIME foi proposto por Ribeiro, Singh e Guestrin (2016) como um método de explicação local que funciona com qualquer tipo de modelo e qualquer tipo de entrada, sem precisar conhecer sua estrutura interna. O método parte do princípio de que, embora um modelo de aprendizado profundo seja globalmente complexo e não-linear, seu comportamento em torno de um exemplo específico pode ser aproximado por um modelo simples e interpretável.

O processo de geração de uma explicação LIME para uma instância de texto ocorre em quatro etapas. Primeiro, o texto original é perturbado pela remoção ou substituição aleatória de palavras, criando um conjunto de versões modificadas da frase. Segundo, cada versão modificada é submetida ao modelo para que ele produza uma predição, permitindo observar se e como a resposta muda conforme as palavras são alteradas. Terceiro, as versões modificadas são ponderadas pela similaridade com o texto original, sendo que versões mais próximas do original recebem peso maior. Quarto, um modelo linear simples é ajustado sobre esse conjunto, e seus coeficientes indicam quais palavras mais influenciaram a classificação (RIBEIRO; SINGH; GUESTLIN, 2016).

Formalmente, o LIME resolve o seguinte problema de otimização para encontrar a explicação $\xi(x)$ de uma instância x :

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (2.1)$$

onde f é o modelo de caixa-preta a ser explicado, G é o conjunto de modelos interpretáveis candidatos, $\mathcal{L}$ é uma função de perda que mede a fidelidade da explicação g ao modelo f na vizinhança de x , π_x é uma função de proximidade que pondera as instâncias perturbadas pela similaridade com x , e $\Omega(g)$ é uma medida de complexidade do modelo explicativo (RIBEIRO; SINGH; GUESTLIN, 2016). O objetivo é encontrar o modelo interpretável mais simples possível que seja localmente fiel ao comportamento do modelo complexo.

A Figura 1 ilustra esse processo: as palavras destacadas indicam sua contribuição positiva ou negativa para a classe predita.

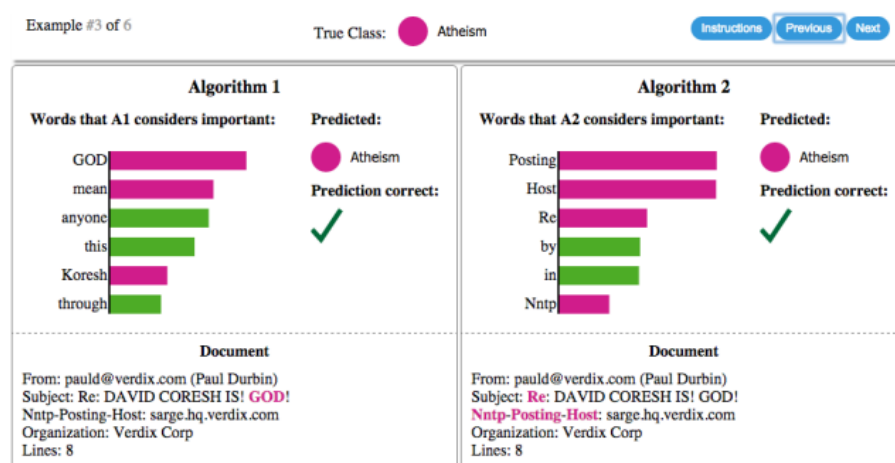


Figura 1 – Exemplo de explicação gerada pelo LIME

Fonte: Ribeiro, Singh e Guestrin (2016)

2.4.2.2 Aplicação em classificação de texto

No contexto de classificação de texto, cada palavra (ou *token*) da entrada é tratada como uma *feature*, ou seja, uma característica binária que indica se está presente ou ausente no texto. A explicação LIME indica quais palavras, quando presentes, aumentam a probabilidade de cada classe, e quais palavras a diminuem. O resultado pode ser visualizado como um destaque colorido sobre o texto original, onde palavras em verde contribuem positivamente para a classe predita e palavras em vermelho contribuem negativamente (MUSTAFA; Hama Saeed, 2025).

Mustafa e Hama Saeed (2025) demonstraram que a aplicação de LIME na classificação de texto não apenas produz explicações úteis para o usuário final, mas também contribui para melhorar o processo de pré-processamento: ao revelar que *stopwords* recebem pesos desproporcionalmente altos nas explicações, o LIME evidenciou a necessidade de sua remoção.

2.4.2.3 Vantagens e limitações

A principal vantagem do LIME é sua generalidade: o método funciona com qualquer modelo e qualquer tipo de entrada sem necessidade de acesso ao interior do modelo. Isso o torna especialmente adequado para modelos de caixa-preta como o BERT, cujos mecanismos internos são de difícil interpretação direta.

Sua principal limitação é a instabilidade das explicações. Por depender de amostragem aleatória de perturbações, execuções diferentes do LIME sobre a mesma instância podem produzir explicações distintas, especialmente quando há colinearidade entre as *features*, ou seja, quando algumas características extraídas do texto são muito parecidas ou carregam informações semelhantes (SALIH et al., 2023). Para mitigar essa instabilidade, recomenda-se executar o LIME múltiplas vezes e analisar a consistência das explicações, ou fixar a semente aleatória para garantir reprodutibilidade.

2.4.3 SHAP — *SHapley Additive exPlanations*

2.4.3.1 Fundamentos — valores de Shapley da teoria dos jogos

SHAP foi proposto por Lundberg e Lee (2017) com base nos valores de Shapley da teoria dos jogos cooperativos. O problema que o SHAP resolve é: dada uma predição específica de um modelo, como distribuir o “crédito” por essa predição de forma justa entre as *features* de entrada?

Na teoria dos jogos, o valor de Shapley de um jogador em um jogo cooperativo é calculado como a média ponderada de quanto cada jogador contribuiu individualmente em todas as combinações possíveis com os demais jogadores. Lundberg e Lee (2017) adaptam esse conceito para o contexto de modelos de aprendizado de máquina: cada *feature* é um

“jogador”, a predição do modelo é o “valor do jogo”, e o valor SHAP de cada *feature* mede o quanto ela contribuiu para a predição, considerando todas as possíveis ordens em que as *features* poderiam ser incluídas.

Formalmente, o valor de Shapley de uma *feature* i é calculado como:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)] \quad (2.2)$$

onde F é o conjunto de todas as *features*, S é um subconjunto que não inclui i , $f(S)$ é a predição do modelo usando apenas as *features* em S , e $f(S \cup \{i\})$ é a predição incluindo a *feature* i (LUNDBERG; LEE, 2017). O fator $\frac{|S|!(|F| - |S| - 1)!}{|F|!}$ garante que todas as ordenações possíveis de inclusão das *features* sejam consideradas com peso igual, assegurando a propriedade de simetria.

A Figura 2 ilustra esse conceito: cada *feature* desloca a predição do valor base até o valor final do modelo.

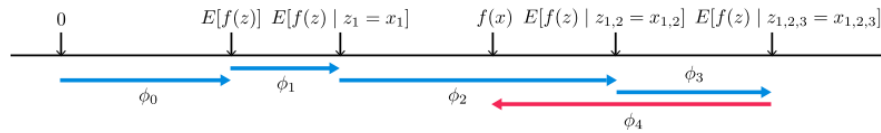


Figura 2 – Representação dos valores SHAP por *feature*

Fonte: Lundberg e Lee (2017)

2.4.3.2 Propriedades matemáticas — eficiência, simetria e ausência

A formulação baseada em valores de Shapley garante ao SHAP três propriedades matemáticas desejáveis que o distinguem de métodos *ad hoc* de atribuição de importância (LUNDBERG; LEE, 2017). A propriedade de eficiência estabelece que a soma dos valores SHAP de todas as *features* iguala a diferença entre a predição do modelo para a instância em questão e o valor base (predição média do modelo sobre o conjunto de dados). A propriedade de simetria estabelece que *features* com contribuições iguais ao modelo recebem valores SHAP iguais, independentemente de sua posição na entrada. A propriedade de ausência estabelece que *features* que não afetam a predição recebem valor SHAP zero.

Essas propriedades conferem ao SHAP uma fundamentação teórica sólida que o LIME não possui, tornando suas explicações mais consistentes e matematicamente justificáveis.

2.4.3.3 KernelSHAP e aplicação em modelos de texto

Calcular o valor exato de Shapley se torna inviável quando o modelo possui muitas *features*, pois seria necessário testar todas as combinações possíveis entre elas, o que cresce exponencialmente. Para resolver isso, o *KernelSHAP* (LUNDBERG; LEE, 2017) estima

esses valores testando apenas uma amostra dessas combinações, tornando o método viável mesmo em modelos de texto, onde cada *token* é tratado como uma *feature*.

Para modelos baseados em *transformers*, o *KernelSHAP* é a variante mais utilizada. O resultado é um vetor de importância por *token* que pode ser visualizado como mapa de calor sobre o texto de entrada, *tokens* com valores SHAP positivos contribuem para a classe predita, e *tokens* com valores negativos contribuem contra ela. O presente trabalho pretende utilizar esses valores SHAP por *token* como pesos dos nós no grafo de co-ocorrência, em conjunto com os valores LIME, permitindo comparação visual entre os dois métodos de explicabilidade.

2.4.4 Comparação entre SHAP e LIME

2.4.4.1 Consistência e estabilidade das explicações

Salih et al. (2023) realizaram uma análise comparativa abrangente entre SHAP e LIME, avaliando suas propriedades teóricas e comportamento empírico em tarefas de classificação. Em termos de consistência, o SHAP demonstra maior estabilidade: por ser baseado em uma formulação matemática única com garantias teóricas, suas explicações são menos sensíveis à aleatoriedade do processo de amostragem. O LIME, por depender de perturbações aleatórias e de um processo de ajuste local, pode produzir explicações distintas para a mesma instância em execuções diferentes.

Os autores concluem que SHAP é preferível quando a consistência e a fundamentação teórica das explicações são prioritárias, enquanto o LIME pode ser preferível quando a velocidade de geração das explicações é mais importante do que a estabilidade.

2.4.4.2 Custo computacional

Em termos de custo computacional, o LIME é mais eficiente que o *KernelSHAP* em textos longos porque trabalha com um número fixo de perturbações. Já o *KernelSHAP* exige uma amostragem muito maior de combinações de *features* para gerar estimativas precisas, o que eleva o tempo de processamento (SALIH et al., 2023). Para textos curtos, como os chamados de suporte de TI, que tipicamente contêm poucas dezenas de palavras, essa diferença de custo é menos pronunciada.

2.4.4.3 Justificativa para uso conjunto neste trabalho

A decisão de implementar e comparar SHAP e LIME neste trabalho, em vez de escolher apenas um dos métodos, é fundamentada em duas razões. Primeiro, a comparação entre os dois métodos produz evidência empírica sobre a qualidade e consistência das explicações geradas para chamados de TI em PT-BR, contribuição metodológica relevante para a literatura sobre XAI em domínios especializados. Segundo, a construção do grafo de co-ocorrência proposto utiliza os valores SHAP e LIME como pesos dos nós, permitindo

avaliar se a escolha do método de XAI afeta a estrutura e a interpretabilidade do grafo resultante.

2.4.5 XAI aplicado a classificação de texto e tickets de TI

2.4.5.1 LIME em classificação multiclasse de texto

Mustafa e Hama Saeed (2025) propuseram um *framework* de classificação de texto que combina PLN e XAI. O sistema inclui pré-processamento com NLTK (*Natural Language Toolkit*), balanceamento de classes com SMOTE (*Synthetic Minority Oversampling Technique*) para lidar com datasets desbalanceados, e validação cruzada estratificada repetida: uma técnica que divide os dados em múltiplos subconjuntos para avaliar o modelo de forma mais robusta, garantindo que todas as classes estejam representadas em cada divisão. O modelo utilizado foi um híbrido de regressão *ridge* com regressão logística. O LIME foi utilizado para validar a explicabilidade das classificações em seis domínios distintos: medicina, esportes, entretenimento, política, tecnologia e negócios. O modelo híbrido atingiu 99% de acurácia após a aplicação do pré-processamento informado pelas explicações LIME.

O trabalho demonstra que a integração de XAI ao *pipeline* de classificação não é apenas uma camada de interpretabilidade adicionada ao final do processo, mas pode também influenciar decisões de pré-processamento e seleção de *features*, melhorando a qualidade do modelo como um todo.

2.4.5.2 SHAP e LIME em ambiente ITSM corporativo

Razma e Jurkus (2026) aplicaram LIME e SHAP em um ambiente ITSM corporativo real, comparando as explicações geradas para o modelo de regressão logística com *TF-IDF* e para o mBERT *fine-tuned*. Os resultados qualitativos das explicações foram significativamente distintos: o modelo linear tendia a destacar palavras-chave literais com alta frequência no corpus, enquanto o BERT destacava termos semanticamente coerentes com o significado do chamado.

Essa diferença qualitativa nas explicações, documentada por Razma e Jurkus (2026), sugere que o BERTimbau, combinado com SHAP e LIME, tende a produzir explicações mais acuradas em termos de importância de *features* e potencialmente mais interpretáveis para analistas de TI sem formação em aprendizado de máquina, hipótese que será verificada empiricamente no presente trabalho, avaliando se os termos destacados fazem sentido para cada categoria de chamado classificada.

2.5 Grafos de Co-ocorrência como Representação Estrutural de Texto

2.5.1 Fundamentos de grafos aplicados a texto

2.5.1.1 Definição de grafo de co-ocorrência — nós, arestas e janela deslizante

Um grafo de co-ocorrência de texto é uma estrutura $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$, onde:

- V representa o conjunto de nós, correspondendo aos *tokens* únicos do texto;
- E representa o conjunto de arestas, conectando pares de *tokens* que co-ocorrem dentro de uma janela deslizante de tamanho w ;
- W representa os pesos das arestas, determinados pela frequência de co-ocorrência entre os *tokens*;
- f_{shap} representa os pesos dos nós calculados pelo método SHAP, indicando a contribuição de cada *token* para a predição do modelo (LUNDBERG; LEE, 2017);
- f_{lime} representa os pesos dos nós calculados pelo método LIME, permitindo comparação visual entre os dois métodos de explicabilidade (RIBEIRO; SINGH; GUESTIN, 2016).

A janela deslizante percorre o texto posição por posição; em cada posição, todos os pares de *tokens* dentro da janela recebem uma aresta ou têm o peso de sua aresta existente incrementado (YAO; MAO; LUO, 2019). Essa representação captura informação estrutural sobre o texto que as representações vetoriais não preservam: dois *tokens* que frequentemente aparecem juntos no mesmo contexto estarão conectados por uma aresta de alto peso no grafo, independentemente de sua posição absoluta na sequência. Para chamados de TI, isso significa que termos técnicos que tendem a co-ocorrer em chamados de uma mesma categoria, como “VPN” e “acesso” em chamados de conectividade, estarão estruturalmente próximos no grafo, com pesos de nós proporcionais à sua importância explicativa calculada por SHAP e LIME.

2.5.1.2 Ponderação de arestas — frequência, PMI

O peso das arestas em um grafo de co-ocorrência pode ser determinado por diferentes métricas. A abordagem mais simples utiliza a frequência bruta de co-ocorrência, ou seja, a contagem de quantas vezes o par de *tokens* co-ocorreu dentro da janela deslizante. Métricas mais sofisticadas, como a Informação Mútua Pontual (PMI, do inglês *Pointwise Mutual Information*), normalizam a frequência de co-ocorrência pelas frequências individuais de cada *token*, favorecendo pares que co-ocorrem mais do que seria esperado pelo acaso (YAO; MAO; LUO, 2019).

No presente trabalho, os pesos das arestas serão determinados pela frequência de co-ocorrência normalizada, enquanto os pesos dos *nós*, que constituem o elemento central da proposta, serão determinados pelos valores SHAP e LIME calculados para cada *token* pelo modelo BERTimbau, permitindo comparar visualmente a importância atribuída por cada método. Essa distinção entre ponderação de arestas (estrutura relacional) e ponderação de nós (relevância explicativa) é a inovação central da camada de visualização proposta.

2.5.2 *TextGCN* — formalização do grafo de co-ocorrência

2.5.2.1 Grafo heterogêneo documento-palavra

Yao, Mao e Luo (2019) propuseram o *TextGCN* (*Text Graph Convolutional Network*), modelo que constrói um único grafo heterogêneo para todo o corpus, conectando documentos e palavras. As arestas documento-palavra recebem pesos baseados em TF-IDF, refletindo a importância de cada palavra para cada documento. As arestas palavra-palavra recebem pesos baseados em PMI calculada sobre co-ocorrências em janela deslizante de tamanho 20 sobre todo o corpus, refletindo a força de associação semântica entre pares de palavras.

Essa estrutura permite que as redes neurais de grafos (GCN) propaguem informação simultaneamente entre documentos e palavras, capturando relações semânticas globais que modelos baseados em sequências não conseguem representar diretamente.

2.5.2.2 Redes neurais de grafos e propagação de informação

As redes neurais de grafos (GNN, do inglês *Graph Neural Networks*) operam sobre grafos propagando informação entre nós vizinhos por meio de operações de agregação e transformação. Em cada camada da rede, a representação de cada nó é atualizada com base nas representações de seus vizinhos na iteração anterior (YAO; MAO; LUO, 2019). Após múltiplas camadas, cada nó possui uma representação que agrega informação de sua vizinhança local de múltiplos saltos.

No contexto deste trabalho, a relevância do *TextGCN* é fundamentalmente teórica: ele formaliza matematicamente a estrutura do grafo de co-ocorrência e demonstra sua utilidade para classificação de texto. O presente trabalho não irá implementar GCN, o grafo de co-ocorrência será utilizado exclusivamente como estrutura de visualização para as explicações XAI, não como componente do modelo de classificação.

2.5.3 Combinação de BERT com grafos de co-ocorrência

2.5.3.1 *Embeddings* BERT como *features* iniciais dos nós

Yang e Cui (2021) propuseram o BEGNN (*BERT-Enhanced Graph Neural Network*), que combina representações contextualizadas do BERT com grafos de co-ocorrência para

classificação de texto. Na arquitetura BEGNN, o BERT gera *embeddings* contextualizados para cada *token* da entrada; esses *embeddings* são utilizados como *features* iniciais dos nós do grafo de co-ocorrência correspondente ao documento. Em seguida, uma GCN propaga informação entre nós vizinhos, produzindo representações enriquecidas pela estrutura do grafo. A representação final do documento é obtida por agregação das representações dos nós.

Essa combinação aborda uma limitação do BERT quando utilizado isoladamente: o mecanismo de atenção do BERT captura relações entre *tokens*, mas essas relações são implícitas nos pesos de atenção e não explicitamente estruturadas como um grafo. O grafo de co-ocorrência fornece uma estrutura explícita que complementa as representações contextualizadas.

2.5.3.2 Resultados em vocabulário técnico especializado

Yang e Cui (2021) avaliaram o BEGNN em múltiplos *datasets* de classificação de texto, demonstrando que a combinação BERT + grafo supera tanto o BERT isolado quanto o *TextGCN*, especialmente em *datasets* com vocabulário técnico especializado. Esse resultado é relevante para o presente trabalho porque chamados de suporte de TI são exatamente o tipo de texto com vocabulário técnico especializado para o qual o BEGNN demonstrou vantagem, sugerindo que a estrutura de grafo de co-ocorrência captura informação complementar às representações do BERTimbau.

2.5.4 Grafo de co-ocorrência combinado com XAI

2.5.4.1 Ponderação de nós por *scores* de explicabilidade

Talebi e Abdolvand (2025) apresentam o trabalho de maior proximidade metodológica com o presente TCC: aplicam RoBERTa combinado com LIME e um grafo de co-ocorrência para detectar *body shaming*, prática de humilhar pessoas com base em sua aparência física, em redes sociais em inglês. No trabalho de Talebi e Abdolvand, o grafo de co-ocorrência é construído com nós representando *tokens* e arestas ponderadas pelos *scores* LIME de cada *token*. Os *tokens* com maior importância LIME produzem arestas de maior peso com seus vizinhos na janela de co-ocorrência.

O presente trabalho adotará uma abordagem estruturalmente análoga, mas com diferenças importantes: utilizará BERTimbau em vez de RoBERTa, aplicará os valores SHAP e LIME como pesos dos nós em vez dos *scores* LIME como pesos de arestas, e o domínio de aplicação será a classificação de chamados de suporte de TI em português brasileiro. A escolha de ponderar os nós em vez das arestas reflete a intenção de destacar visualmente os *tokens* individualmente mais relevantes para a predição, enquanto as arestas manterão sua função de representar a estrutura de co-ocorrência do texto.

2.5.4.2 Visualização interativa como camada de auditabilidade

A literatura aponta que a combinação de grafos de co-ocorrência com *scores* de explicabilidade pode ser exibida em interfaces interativas que tornam o processo decisório auditável. Nessa abordagem, os nós do grafo são dimensionados proporcionalmente à importância de cada *token* calculada pelo método XAI, e as arestas são ponderadas pela frequência de co-ocorrência, produzindo uma representação visual que combina estrutura relacional do texto com relevância explicativa do modelo (TALEBI; ABDOLVAND, 2025).

Essa camada de visualização tem o potencial de transformar a saída opaca do modelo em uma representação auditável, endereçando diretamente o problema identificado por Zemp (2021) e Razma e Jurkus (2026): analistas de suporte precisam compreender por que o modelo tomou determinada decisão para confiar no sistema e adotá-lo em produção. A solução proposta neste trabalho adota essa abordagem, cuja implementação é detalhada no Capítulo 3.

2.6 Síntese da Literatura e Posicionamento do Trabalho

2.6.1 Lacunas identificadas no estado da arte

2.6.1.1 Ausência de XAI em sistemas de classificação de tickets em produção

O trabalho de Zemp (2021) demonstra que sistemas de classificação de chamados de TI com alta acurácia são tecnicamente viáveis e já foram implantados em produção. No entanto, a ausência de mecanismos de explicabilidade limita a adoção e a auditabilidade desses sistemas. Os agentes de suporte recebem previsões sem justificativa, o que dificulta a identificação de erros sistemáticos e a construção de confiança no sistema. Nenhum dos trabalhos de classificação de chamados de TI identificados no *screening* incorpora uma camada de visualização gráfica para explicações XAI.

2.6.1.2 Carência de soluções robustas para PT-BR em *helpdesk*

Os trabalhos de Yokota (2025) e Pereira et al. (2023) evidenciam que o estado da arte em classificação de chamados de suporte em português brasileiro ainda é incipiente. Os melhores resultados reportados com modelos simples (75% de precisão média com ML tradicional e F1-macro insatisfatório com NB e MLP) são inferiores a modelos baseados em *transformers* já alcançados em inglês. Não foi identificado nenhum trabalho nacional que aplique BERTimbau com XAI a chamados de suporte de TI.

2.6.1.3 Custo-benefício do BERT sem camada de explicabilidade

Wahba (2023) questiona o uso de BERT para classificação de texto em domínios especializados, argumentando que o custo computacional adicional não se justifica diante da

pequena vantagem sobre o SVM em textos técnicos com baixa polissemia. Esse argumento permanece válido enquanto o único critério de avaliação for a acurácia. O presente trabalho responde a esse argumento demonstrando que o custo do BERTimbau se justifica quando a camada de XAI e grafo de co-ocorrência é adicionada, transformando a saída opaca do modelo em algo auditável e confiável para os analistas, critério que modelos lineares não conseguem satisfazer com a mesma qualidade de explanação.

2.6.2 Direcionamento para a proposta

Diante das lacunas identificadas: ausência de XAI em sistemas de classificação de chamados em produção, carência de soluções robustas para PT-BR em *helpdesk* de TI, e a questão do custo-benefício do BERT sem explicabilidade, o presente trabalho investiga um *pipeline* que combina BERTimbau, SHAP, LIME e grafo de co-ocorrência ponderado para classificação de chamados em PT-BR com explicabilidade visual. A descrição detalhada da arquitetura, do *dataset*, dos experimentos e da interface que será desenvolvida é apresentada no Capítulo 3.

2.7 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi realizada nas bases Google Scholar, IEEE Xplore, arXiv e SciELO, utilizando as palavras-chave “BERT”, “Grafos”, “XAI”, “PLN”, “Help Desk”, “Chamados de TI”, “SHAP”, “LIME”, “Tickets”, “Support”, “Pipeline”, “Classificação”, “Caixa Preta” e “Knowledge Graph”, entre outras. Foram selecionados artigos publicados entre 2021 e 2026, em português e inglês, que tivessem relação direta com o tema abordado.

2.8 Fundamentos

Esta seção apresenta os conceitos técnicos fundamentais que sustentam a proposta deste trabalho. O objetivo é fornecer ao leitor as definições necessárias para compreender a solução proposta antes de sua descrição detalhada.

2.8.1 Processamento de Linguagem Natural

Processamento de Linguagem Natural (PLN) é o campo da inteligência artificial dedicado ao desenvolvimento de métodos computacionais para compreender, interpretar e gerar linguagem humana. No contexto deste trabalho, PLN é utilizado para representar chamados de suporte técnico como vetores numéricos que um modelo de aprendizado de máquina possa processar, e para extrair informações semânticas dos textos submetidos pelos usuários (DEVLIN et al., 2019).

2.8.2 Modelos de Linguagem Baseados em *Transformer*

Modelos de linguagem baseados na arquitetura *Transformer* (VASWANI et al., 2017) aprendem representações contextualizadas de texto por meio de mecanismos de atenção que consideram simultaneamente todas as posições de uma sequência de entrada. O BERT (DEVLIN et al., 2019) é o modelo desta família mais amplamente adotado para tarefas de classificação de texto. O BERTimbau (SOUZA; NOGUEIRA; LOTUFO, 2020) é uma variante do BERT pré-treinada especificamente sobre corpora em português brasileiro, sendo o modelo adotado neste trabalho.

2.8.3 Inteligência Artificial Explicável

Inteligência Artificial Explicável (XAI) refere-se ao conjunto de métodos e técnicas que permitem compreender e auditar as decisões de modelos de aprendizado de máquina (SALIH et al., 2023). Dois métodos *post-hoc* serão utilizados neste trabalho:

- **LIME** (RIBEIRO; SINGH; GUESTRIN, 2016): explica predições individuais ajustando um modelo linear simples em torno de perturbações da entrada original, indicando quais palavras mais contribuíram para a classificação.
- **SHAP** (LUNDBERG; LEE, 2017): atribui a cada palavra da entrada um valor de importância calculado com base na teoria dos jogos cooperativos, com garantias matemáticas de consistência e equidade na distribuição de crédito.

2.8.4 Grafo de Co-ocorrência

Um grafo de co-ocorrência é uma estrutura $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$ na qual os nós V representam os termos únicos de um texto e as arestas E conectam pares de termos que aparecem juntos dentro de uma janela deslizante de tamanho fixo (YAO; MAO; LUO, 2019). Neste trabalho, essa estrutura será utilizada como camada de visualização das explicações XAI: os pesos dos nós serão determinados pelos valores SHAP e LIME de cada *token*, e os pesos das arestas pela frequência de co-ocorrência, produzindo uma representação visual que combina estrutura relacional do texto com relevância explicativa do modelo.

2.8.5 Sistema de *Help Desk* e Triagem de Chamados

Um sistema de *help desk* é uma plataforma que centraliza e gerencia solicitações de suporte técnico submetidas pelos usuários de uma organização. A triagem de chamados consiste em classificar cada solicitação em uma categoria predefinida e encaminhá-la à equipe responsável pela resolução. A automação desse processo, substituindo ou auxiliando a triagem manual, é o problema central abordado por este trabalho (ZEMP, 2021).

2.9 Trabalhos relacionados

Esta seção analisa três trabalhos que enfrentaram o desafio de automatizar a triagem de chamados de suporte de TI, servindo de base comparativa para a evolução proposta neste TCC.

2.9.1 Trabalho 1: Ticket Automation: An Insight into Current Research

Zemp (2021) abordou o problema da triagem automática num ambiente corporativo com mais de um milhão de chamados. Para resolvê-lo, o autor comparou modelos CNN, BiLSTM e RoBERTa. A metodologia envolveu o treino em larga escala num *dataset* real de um banco suíço. Os resultados mostraram uma acurácia de 92%. No entanto, a solução apresentou limitações na aceitação pelos utilizadores finais devido à natureza de "caixa-preta" do modelo. O presente trabalho diferencia-se por focar na explicabilidade via SHAP e Grafos, permitindo que os analistas de TI entendam o "porquê" da classificação.

2.9.2 Trabalho 2: Machine Learning for Classification of IT Support Tickets

Wahba (2023) enfrentou o mesmo desafio de classificação de *tickets*, mas propôs uma abordagem focada em eficiência para domínios especializados. A solução utilizou um modelo híbrido (HOOM) que combina SVM linear com classificadores *online*. A metodologia demonstrou que em textos técnicos, modelos mais simples podem ser competitivos. Os resultados mostraram boa performance com baixo custo computacional. A limitação identificada é que essa abordagem perde as nuances contextuais que modelos baseados em *Transformers* capturam. O presente trabalho pretende adotar o BERTimbau, específico para PT-BR, combinado com visualização de grafos para suprir a falta de contexto de modelos lineares.

2.9.3 Trabalho 3: Classificação hierárquica de textos aplicado ao contexto chamados de suporte

Yokota (2025) investigou a classificação de chamados especificamente em português brasileiro, utilizando modelos clássicos como *Naive Bayes* e MLP. A metodologia focou na estrutura hierárquica das categorias de TI. Os resultados apresentaram F1-macro insatisfatório para uso em produção. A lacuna principal foi a incapacidade de modelos simples lidarem com o jargão técnico do suporte em português. O presente trabalho diferencia-se pelo uso de uma arquitetura de estado da arte (BERTimbau) treinada especificamente no nosso idioma, superando as limitações semânticas dos modelos testados por Yokota.

Tabela 1 – Comparação entre trabalhos relacionados

Critério	Zemp (2021)	Wahba (2023)	Yokota (2025)	Este trabalho
Objetivo	Classificação	Eficiência	Classificação PT-BR	Classificação + XAI
Tecnologia	RoBERTa / CNN	SVM / HOOM	Naive Bayes / MLP	BERTimbau + Grafos
Explicabilidade	Não	Intrínseca	Limitada	SHAP / LIME / Grafos
Limitação	Caixa-Preta	Baixo Contexto	Baixa Performance	Custo Computacional

Fonte: Elaborado pelos autores (2026)

2.10 Hipótese de Pesquisa

A utilização de modelagem em grafos como camada de explicabilidade visual para o modelo BERT aumenta a auditabilidade e a confiança dos gestores de TI no processo de classificação automatizada, mitigando a percepção de 'caixa-preta' sem a necessidade de intervenção manual na triagem.

2.11 Métricas de Avaliação

A avaliação do desempenho do modelo será realizada por meio de métricas amplamente utilizadas em tarefas de classificação textual:

- **Acurácia (Accuracy)** – proporção de classificações corretas realizadas pelo modelo sobre o total de instâncias avaliadas.
- **Precisão (Precision)** – proporção de classificações corretas entre os casos previstos para uma determinada categoria, medindo a capacidade do modelo de evitar falsos positivos.
- **Revocação (Recall)** – capacidade do modelo de identificar corretamente os chamados pertencentes a cada categoria, medindo a capacidade de evitar falsos negativos.
- **F1-score** – média harmônica entre precisão e revocação, expressa pela fórmula $F1 = 2 \cdot \frac{\text{Precisão} \times \text{Revocação}}{\text{Precisão} + \text{Revocação}}$. Será utilizada a variante **F1-macro**, que calcula o F1-score para cada categoria individualmente e tira a média, tratando todas as categorias com peso igual independentemente de seu tamanho, métrica especialmente importante em datasets desbalanceados.
- **ROC-AUC** (*Receiver Operating Characteristic - Area Under the Curve*) – mede a capacidade do modelo de distinguir entre classes ao longo de diferentes limiares de decisão. O valor varia entre 0 e 1, sendo que 1 representa um classificador perfeito e 0,5 representa um classificador aleatório. Em cenários multiclasse, será utilizada a

variante **ROC-AUC macro**, que agrega as curvas ROC de cada categoria. Essa métrica permite análises complementares, como a avaliação do impacto de diferentes combinações de hiperparâmetros sobre a capacidade discriminativa do modelo.

- **Matriz de confusão** – representação tabular que detalha os tipos de erros cometidos pelo modelo durante a classificação, permitindo identificar padrões sistemáticos de confusão entre categorias semanticamente próximas.

3 Desenvolvimento

Neste capítulo, apresenta-se a solução proposta para o problema identificado, bem como o cronograma de trabalho previsto para a implementação completa do projeto no TCC2.

3.1 Solução proposta

3.1.0.1 Visão geral da solução

A solução proposta tem como objetivo central classificar automaticamente chamados de suporte técnico de forma transparente e auditável, utilizando técnicas de Processamento de Linguagem Natural (PLN) e Inteligência Artificial Explicável (XAI). O diferencial da proposta não é apenas a classificação automática, mas a capacidade de explicar cada decisão: o analista poderá identificar quais termos do chamado influenciaram a classificação e por quê.

Para isso, serão utilizadas duas técnicas complementares de explicabilidade: o SHAP, que atribui a cada termo do chamado um valor numérico indicando o quanto ele contribuiu para a classificação, e o LIME, que gera explicações locais testando versões modificadas do texto. Os valores SHAP e LIME serão utilizados como pesos dos nós em um grafo de co-ocorrência $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$, produzindo uma visualização gráfica que combina a estrutura relacional do texto com a relevância explicativa de cada termo segundo ambos os métodos.

O sistema será desenvolvido para atender principalmente gestores e analistas de TI, que poderão utilizá-lo durante o processo de triagem para acompanhar de forma transparente e auditável o resultado da classificação automática.

3.1.1 Protocolo experimental

O experimento será estruturado em quatro cenários comparativos, conforme descrito na Tabela 2, com o objetivo de isolar a contribuição de cada componente do *pipeline*:

Tabela 2 – Cenários experimentais para avaliação do sistema

Cenário	Modelo	XAI	Objetivo
A	SVM + TF-IDF	Nenhum	Baseline clássico
B	BERTimbau	Nenhum	Ganho do transformer
C	BERTimbau	SHAP + LIME	Qualidade das explicações
D	BERTimbau	SHAP + LIME + grafo	Auditabilidade visual

Fonte: Elaborado pelos autores (2026)

A comparação entre os cenários A e B avalia o ganho de desempenho do BERTimbau sobre o *baseline* clássico — respondendo ao argumento de [Wahba \(2023\)](#) sobre o custo-benefício dos *transformers* em domínios especializados. A comparação entre os cenários C e D avalia se a camada de grafo de co-ocorrência oferece auditabilidade superior à exibição isolada de *scores* de importância — hipótese central deste trabalho.

3.1.1.1 Protótipos da interface

Durante a etapa inicial do projeto foram elaborados protótipos de baixa e média fidelidade com o objetivo de validar a organização visual da interface, o fluxo de navegação e a disposição das funcionalidades relacionadas à explicabilidade do modelo.

A etapa de prototipação de baixa fidelidade teve início com a elaboração individual de diferentes propostas de interface por cada integrante do grupo, permitindo que distintas ideias e abordagens fossem exploradas inicialmente. Posteriormente, as principais características de cada proposta foram analisadas e consolidadas em uma versão única do protótipo que passou a servir como base para a continuidade do projeto.

Os protótipos de baixa fidelidade foram utilizados para estruturar os principais componentes da aplicação, definindo a posição dos menus, área de inserção de chamados, painel de resultados e espaço destinado às visualizações explicativas.

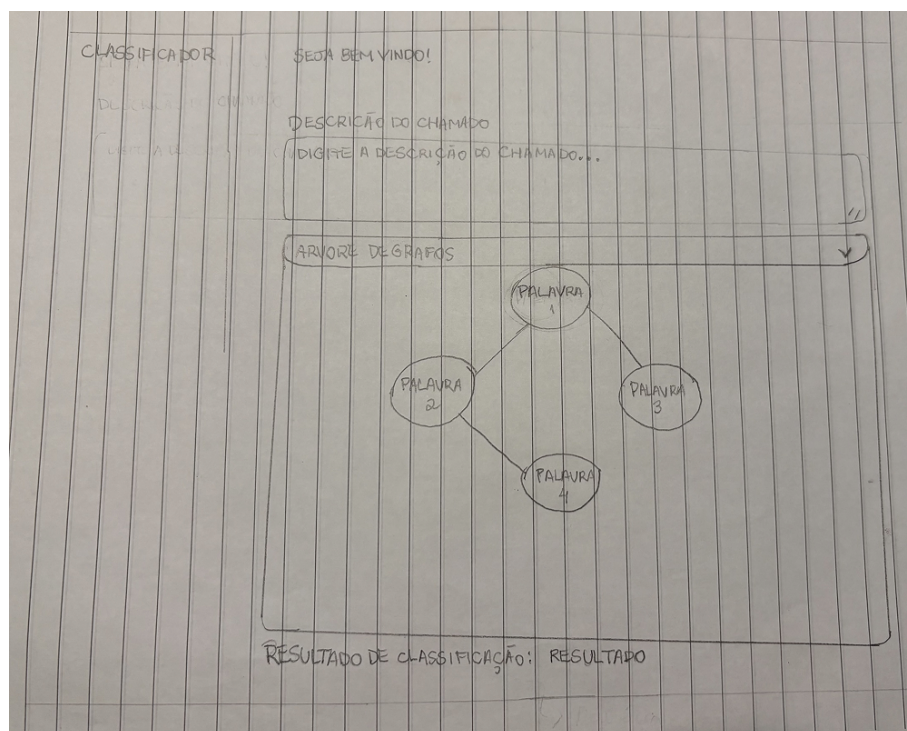


Figura 3 – Protótipo de baixa fidelidade da interface por Agatha Barbosa

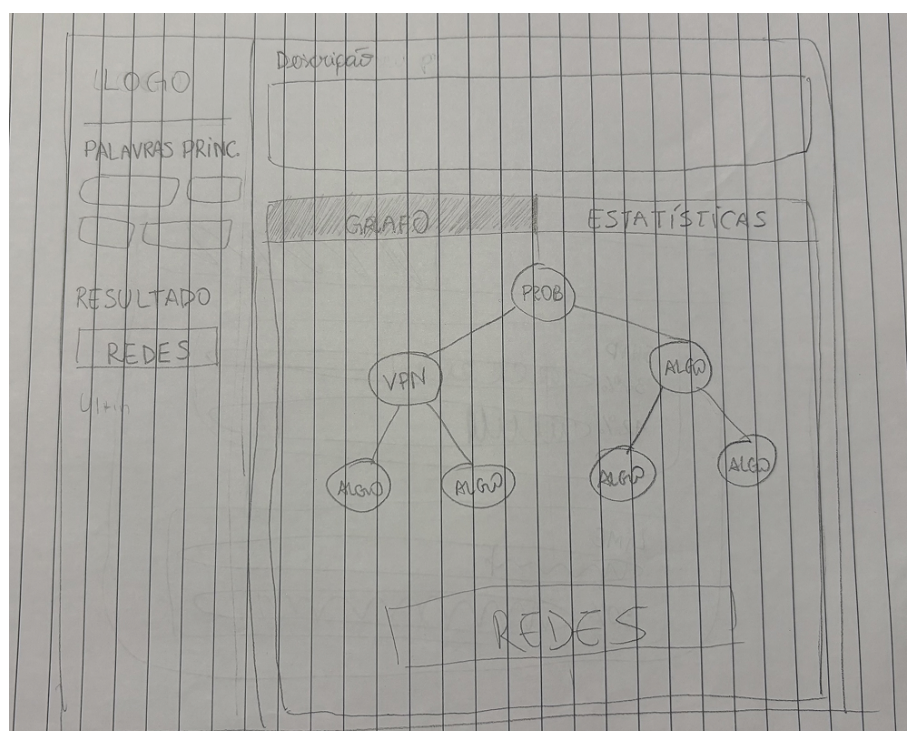


Figura 4 – Protótipo de baixa fidelidade da interface por Bruna Kinjo

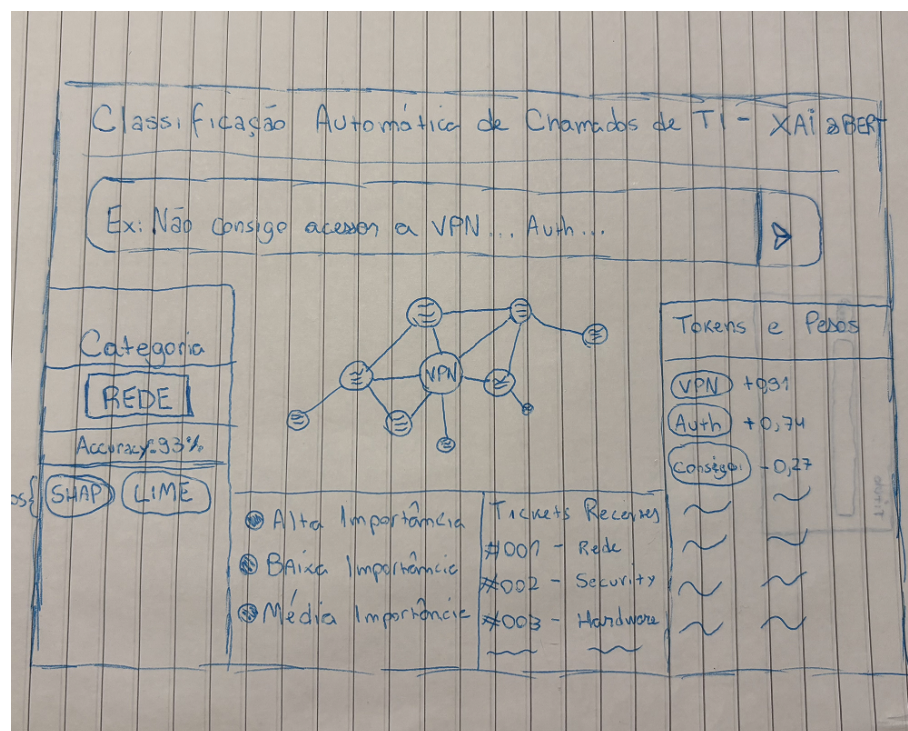


Figura 5 – Protótipo de baixa fidelidade da interface por Kayke Ahrens

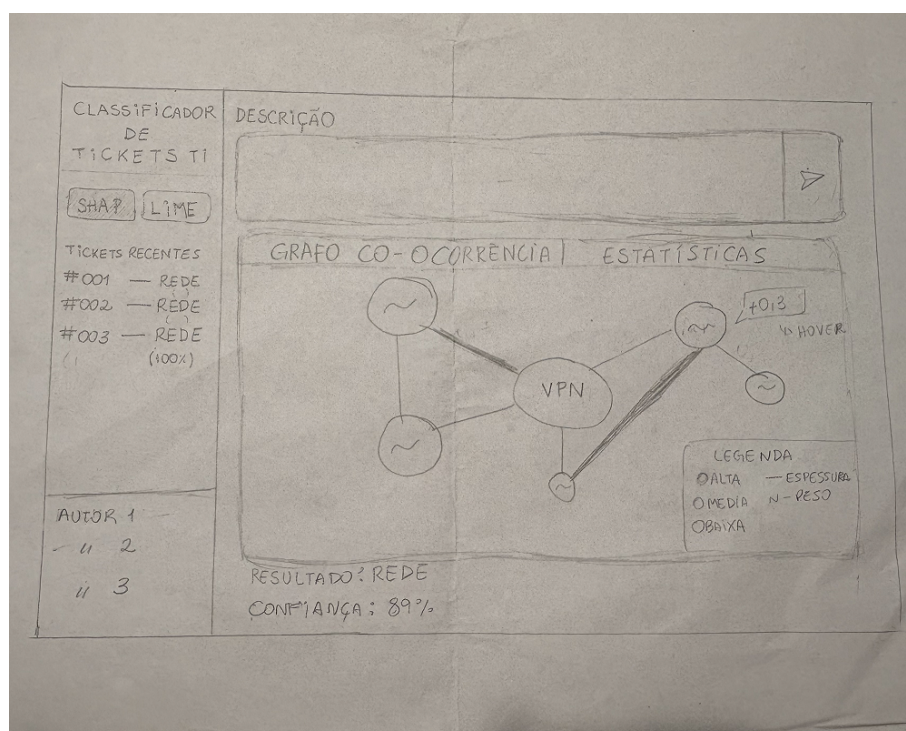


Figura 6 – Protótipo de baixa fidelidade da interface final - Tela do grafo

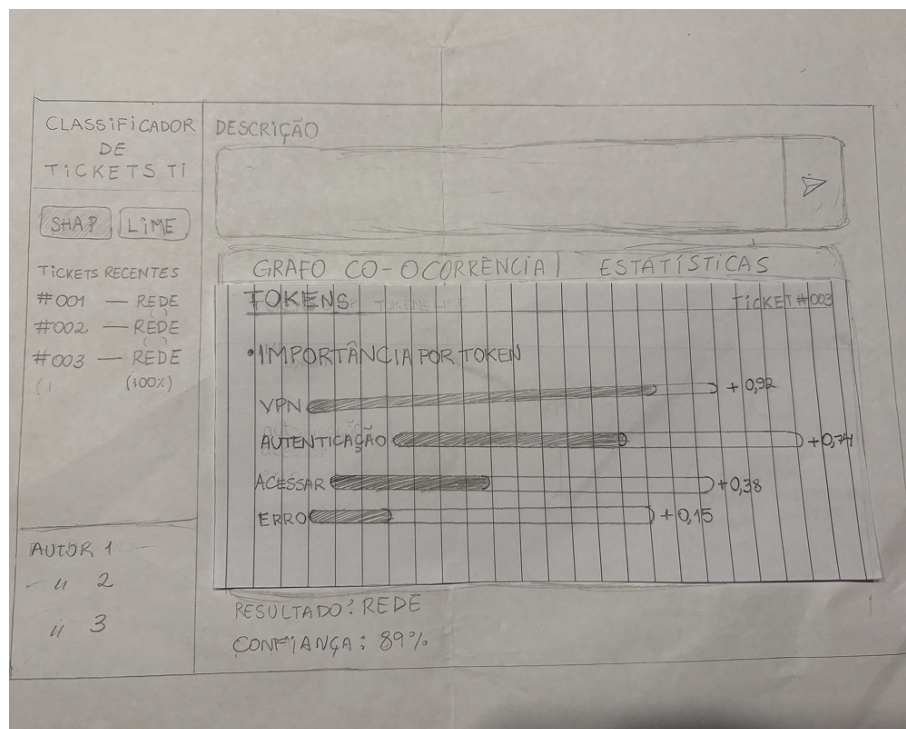


Figura 7 – Protótipo de baixa fidelidade da interface final - Tela das estatísticas

Posteriormente, foram elaborados protótipos de média fidelidade com maior nível de detalhamento visual, com foco em validar como as explicações geradas pelo modelo seriam apresentadas de forma compreensível para os analistas de TI.

A área central do sistema será dedicada às explicações da classificação, organizada em duas abas principais, ambas voltadas para tornar o processo decisório do modelo auditável:

- **Grafo de Co-Ocorrência:** representação visual das relações entre os *tokens* do chamado, onde o tamanho de cada nó será proporcional ao valor SHAP e LIME do termo, indicando sua importância para a classificação, e a espessura das conexões indicará a frequência de co-ocorrência entre os *tokens*;
- **Estatísticas:** painel que apresentará numericamente a importância individual de cada *token* calculada pelos métodos SHAP e LIME, permitindo ao analista comparar e validar as contribuições de cada termo na decisão do modelo.

A escolha por disponibilizar tanto visualizações gráficas quanto estatísticas numéricas busca garantir que analistas com diferentes perfis consigam auditar as decisões do modelo, seja por meio de padrões visuais no grafo ou por comparações quantitativas entre os valores de importância dos *tokens*.

Na lateral esquerda do sistema encontra-se a seção “Tickets recentes”, responsável por apresentar classificações realizadas anteriormente juntamente com seus respectivos níveis de confiança. Esse recurso permitirá consultar rapidamente chamados processados anteriormente e auxiliará na análise das classificações realizadas.

Os protótipos desenvolvidos tiveram como principal objetivo verificar se os recursos de explicabilidade seriam apresentados de forma intuitiva, clara e acessível aos analistas de TI.

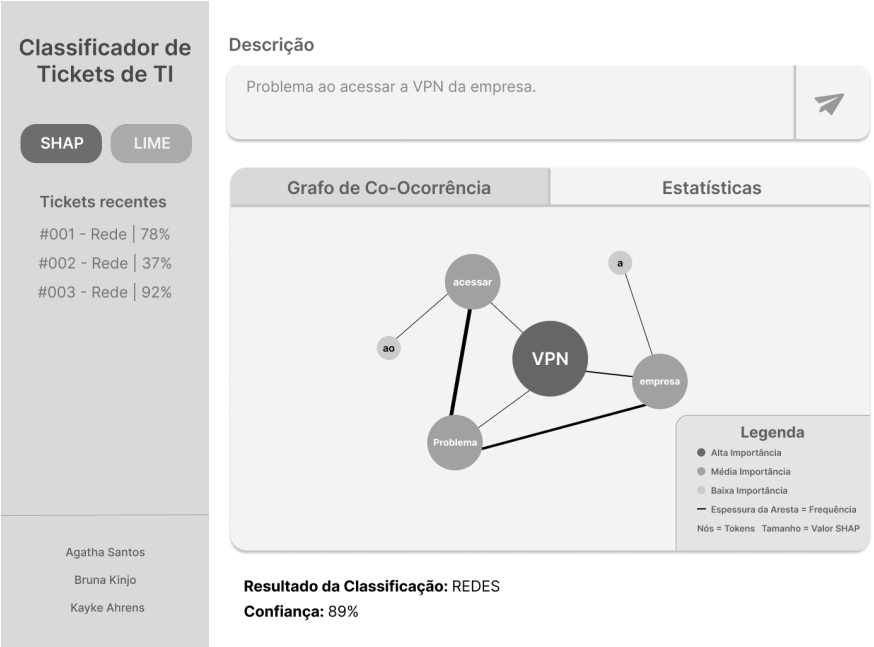


Figura 8 – Protótipo de média fidelidade - visualização do grafo de co-ocorrência



Figura 9 – Protótipo de média fidelidade - visualização estatística dos *tokens*

3.1.1.2 Arquitetura da solução

A arquitetura da solução será organizada em três camadas principais:

- Camada de apresentação (*front-end*);
- Camada de processamento;
- Camada de dados.

A camada de apresentação será responsável pela interação com o usuário final, disponibilizando os formulários de inserção de chamados, exibição das classificações, visualização dos grafos e acesso ao histórico de *tickets* processados.

A camada de processamento será responsável pela execução das etapas de pré-processamento textual, inferência do modelo BERTimbau, cálculo das explicações utilizando SHAP e LIME, além da construção dos grafos de co-ocorrência.

Já a camada de dados será responsável pelo armazenamento das informações relacionadas aos chamados classificados e aos resultados gerados pelo sistema.

O fluxo geral de funcionamento da aplicação seguirá as seguintes etapas:

1. **Pré-processamento** — o texto do chamado é submetido para tokenização, remoção de *stopwords* e normalização. O texto limpo é passado ao tokenizador do BERTimbau, que o converte em *tokens* com seus respectivos IDs e máscaras de atenção.
2. **Classificação com BERTimbau** — o modelo BERTimbau com *fine-tuning* processa a sequência de *tokens* e produz, a partir da representação do *token* [CLS], as probabilidades de pertencimento a cada categoria. A categoria com maior probabilidade é selecionada como predição, e o índice de confiança corresponde a essa probabilidade máxima.
3. **Cálculo de explicabilidade** — sobre o mesmo texto de entrada, são calculados os valores SHAP e as importâncias LIME. Cada *token* recebe um valor numérico que representa sua contribuição para a predição: valores positivos indicam contribuição para a classe predita, e valores negativos indicam contribuição contrária.
4. **Construção do grafo de co-ocorrência** — os *tokens* do chamado são representados como nós de um grafo $G = (V, E, W, f_{\text{shap}}, f_{\text{lime}})$. As arestas conectam pares de *tokens* que co-ocorrem dentro de uma janela deslizante de tamanho fixo, com pesos proporcionais à frequência de co-ocorrência. Os pesos dos **nós** são determinados pelos valores SHAP e LIME calculados na etapa anterior — *tokens* com maior importância SHAP geram nós visualmente maiores na interface. O grafo é serializado em JSON com a estrutura `{nodes, edges}` e enviado ao *frontend*.
5. **Apresentação ao analista** — a interface recebe o JSON, renderiza o grafo e exibe a categoria predita, o índice de confiança, os *tokens* mais relevantes destacados por cor e os valores SHAP e LIME comparativos.

3.1.1.3 Tecnologias e ferramentas

As tecnologias listadas a seguir representam as opções inicialmente consideradas para o desenvolvimento do sistema, podendo ser ajustadas conforme as decisões técnicas tomadas ao longo da implementação.

Interface e visualização:

- React.js;
- CSS;
- Biblioteca para visualização de grafos.

Processamento e Inteligência Artificial:

- Python;
- BERTimbau;
- SHAP;
- LIME;
- Bibliotecas de PLN e manipulação de grafos.

Ambiente de desenvolvimento:

- Git e GitHub;
- Jupyter Notebook.

3.1.1.4 Funcionalidades principais

Entre as principais funcionalidades previstas para o sistema estão:

• Funcionalidades de explicabilidade:

- Construção de grafos de co-ocorrência;
- Visualização das explicações do modelo.

• Funcionalidades de classificação e suporte:

- Classificação automática de chamados de TI;
- Exibição da categoria prevista;
- Apresentação do nível de confiança da classificação;
- Histórico simplificado de *tickets* classificados.

3.1.1.5 Metodologia de implementação

A implementação do projeto seguirá uma abordagem iterativa baseada em prototipação e validações incrementais.

Inicialmente foram realizados levantamento bibliográfico, definição dos requisitos e elaboração dos protótipos da interface, etapas já concluídas nesta fase do projeto. Em seguida serão implementados os módulos de classificação automática, explicabilidade e visualização gráfica.

Posteriormente serão realizados testes de integração entre os componentes e ajustes relacionados à usabilidade e apresentação das explicações geradas pelo sistema.

3.1.1.6 Dataset e fine-tuning

O modelo BERTimbau será ajustado utilizando um conjunto de chamados de dados reais obtidos junto à Aché e Bayer Brasil, mediante acordo de confidencialidade e anonimização.

O processo de *fine-tuning* seguirá uma abordagem supervisionada de classificação textual, utilizando divisão entre conjuntos de treino, validação e teste. Também serão aplicadas técnicas de balanceamento de classes e validação de desempenho para reduzir problemas relacionados a *overfitting*, fenômeno que ocorre quando o modelo aprende os dados de treino com precisão excessiva e perde a capacidade de generalizar para novos exemplos, e ao desbalanceamento do *dataset*.

3.1.1.7 Critérios de validação

O sucesso da solução será avaliado sob duas perspectivas complementares.

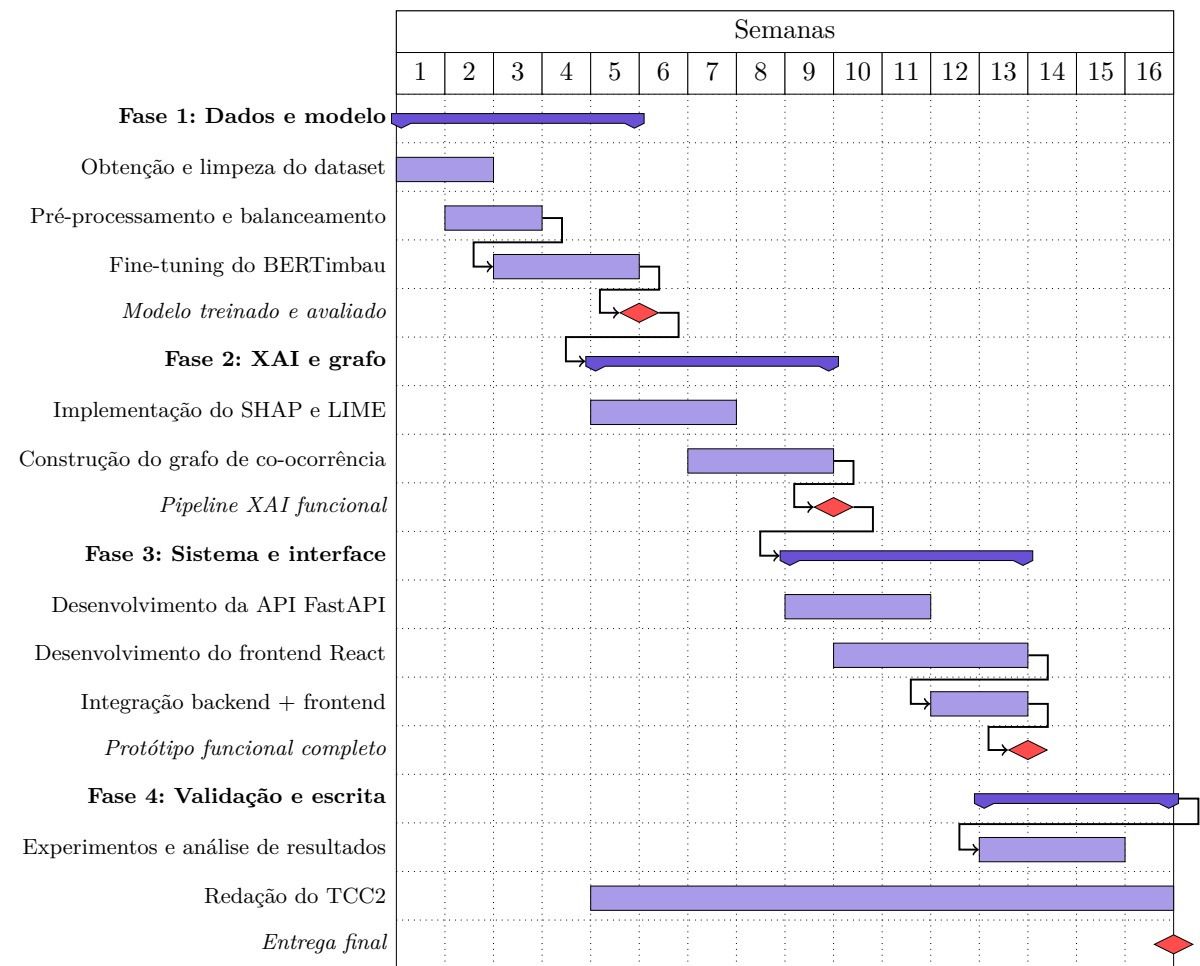
A primeira perspectiva, **quantitativa**, avalia o desempenho do modelo de classificação por meio das seguintes métricas: acurácia, precisão, revocação, F1-score macro para lidar com possível desbalanceamento entre categorias, ROC-AUC macro para avaliar a capacidade do modelo de distinguir entre as categorias ao longo de diferentes limiares de decisão e matriz de confusão para análise de erros sistemáticos. O BERTimbau será considerado satisfatório se superar o *baseline* SVM em pelo menos 5 pontos percentuais de F1-macro.

A segunda perspectiva, **qualitativa**, avalia a utilidade das explicações geradas. Serão analisadas: a concordância entre as explicações SHAP e LIME para os mesmos chamados, e a coerência semântica dos *tokens* destacados no grafo em relação à categoria predita, verificando se os termos com maior importância SHAP e LIME correspondem intuitivamente ao tipo de problema descrito no chamado.

3.2 Cronograma de trabalho

O cronograma apresentado a seguir organiza as atividades previstas para o TCC2 de forma simplificada.

Figura 10 – Cronograma de atividades – Diagrama de Gantt



Fonte: Elaborado pelos autores (2026)

As atividades poderão sofrer ajustes conforme a evolução do projeto e os resultados obtidos durante a implementação e validação da solução.

Referências

BORGES, B. R. *Análise comparativa de algoritmos de classificação de texto*. Dissertação (Trabalho de Conclusão de Curso – Sistemas de Informação) — Universidade Federal de Uberlândia, 2024. Disponível em: <<https://repositorio.ufu.br/handle/123456789/43701>>. 22

DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. [s.n.], 2019. Disponível em: <<https://arxiv.org/abs/1810.04805>>. 18, 19, 20, 33, 34

HARUN, N. A.; HUSPI, S. H.; IAHAD, N. A. Question classification framework for helpdesk ticketing support system using machine learning. *International Journal on Informatics and Communication Technology (IJIC)*, v. 13, n. 1, 2023. Disponível em: <<https://ijic.utm.my/index.php/ijic/article/view/388>>. 14

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. v. 30. Disponível em: <<https://arxiv.org/abs/1705.07874>>. 10, 25, 26, 29, 34

MIKOLOV, T. et al. Efficient estimation of word representations in vector space. In: *International Conference on Learning Representations (ICLR)*. [s.n.], 2013. Disponível em: <<https://arxiv.org/abs/1301.3781>>. 17

MUSTAFA, S.; Hama Saeed, M. Empowering text classification with NLP and explainable AI for enhanced interpretability. *Journal of Electrical Systems and Information Technology*, Springer, v. 12, n. 81, 2025. Disponível em: <<https://link.springer.com/article/10.1186/s43067-025-00273-2>>. 25, 28

PARAMESH, S. P.; SHREEDHARA, K. S. *A Deep Learning Based IT Service Desk Ticket Classifier Using CNN*. 2022. ResearchGate. Disponível em: <<https://www.researchgate.net/publication/366986777>>. 15

PEREIRA, L. S. B. et al. Machine learning for classification of IT support tickets. In: *International Conference on Cyber Management and Engineering (CyMaEn)*. IEEE, 2023. Disponível em: <<https://www.researchgate.net/publication/368906495>>. 14, 15, 32

RAZMA, A.; JURKUS, R. AI-based classification of IT support requests in enterprise service management systems. *Systems*, MDPI, v. 14, n. 223, 2026. Disponível em: <<https://doi.org/10.3390/systems14020223>>. 10, 13, 17, 22, 23, 28, 32

RIBEIRO, M. T.; SINGH, S.; GUESTRIN, C. "why should I trust you?": Explaining the predictions of any classifier. In: *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016. Disponível em: <<https://arxiv.org/abs/1602.04938>>. 10, 23, 24, 29, 34

SALIH, A. et al. A perspective on explainable artificial intelligence methods: SHAP and LIME. *Advanced Intelligent Systems*, Wiley, 2023. Disponível em: <<https://arxiv.org/abs/2305.02012>>. 22, 23, 25, 27, 34

- SOUZA, F.; NOGUEIRA, R.; LOTUFO, R. BERTimbau: Pretrained BERT models for Brazilian Portuguese. In: *Brazilian Conference on Intelligent Systems (BRACIS)*. Springer, 2020. Disponível em: <<https://www.researchgate.net/publication/345395208>>. 21, 34
- TALEBI, S.; ABDOLVAND, N. Building safer social spaces: Body shaming detection using RoBERTa, LIME and term co-occurrence graph. *International Journal of Web Research (IJWR)*, 2025. Disponível em: <<https://doaj.org/article/3a3f1e1beed847909118aac97aacce05>>. 31, 32
- VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. v. 30. Disponível em: <<https://arxiv.org/abs/1706.03762>>. 10, 18, 19, 34
- WAHBA, Y. M. *A Hybrid Continual Machine Learning Model for Efficient Hierarchical Classification of IT Support Tickets in the Presence of Class Overlap*. Tese (Tese de Doutorado em Ciência da Computação) — University of Western Ontario, 2023. Disponível em: <<https://ir.lib.uwo.ca/etd/9192>>. 14, 16, 17, 32, 35, 39
- YANG, X.; CUI, Y. BERT-enhanced text graph neural network for classification. *Entropy*, MDPI, v. 23, n. 1536, 2021. Disponível em: <<https://www.mdpi.com/1099-4300/23/11/1536>>. 30, 31
- YAO, L.; MAO, C.; LUO, Y. Graph convolutional networks for text classification. In: *AAAI Conference on Artificial Intelligence*. [s.n.], 2019. Disponível em: <<https://arxiv.org/abs/1809.05679>>. 29, 30, 34
- YOKOTA, B. *Aprendizado de máquina baseado em Classificação hierárquica de textos aplicado ao contexto chamados de suporte*. Dissertação (MBA em Inteligência Artificial e Big Data) — Universidade de São Paulo – USP/ICMC, 2025. Disponível em: <https://bdta.abcd.usp.br/directbitstream/43ebe5d7-98d6-4357-915e-71486db593d5/Beatriz_Yokota.pdf>. 11, 15, 32, 35
- ZEMP, M. *Text Classification of Service Desk Tickets*. Dissertação (Master Thesis – MAS Data Science) — Zurich University of Applied Sciences (ZHAW), 2021. Disponível em: <https://www.zhaw.ch/storage/shared/upload/MAS21_Ticket_Classification_Zemp.pdf>. 10, 13, 16, 22, 23, 32, 34, 35